Table des matières

Chapitre 1 : L'écosystème Java **4**
1.1 Java en bref 4
1.2 Du code source à l'exécution 4
1.3 La JVM : « Write once, run anywhere » 5
1.4 JDK, JRE, JVM : qui contient quoi ? 5
1.5 Le ramasse-miettes (Garbage Collector) 6
1.6 Installer le JDK 7
1.7 Votre premier programme 7

Chapitre 2 : Ce qui fait la force de Java **9**
2.1 Les six caractéristiques 9

Chapitre 3 : Types, variables et modificateurs **11**
3.1 Les 8 types primitifs 11
3.2 Valeur ou référence ? 12
3.3 Trois sortes de variables 13
3.4 Variable static ou variable d'instance ? 13
3.5 Exercice : trouvez les trois erreurs 14
3.6 Modificateurs : accès et comportement 14

Chapitre 4 : Les opérateurs **16**
4.1 Opérateurs arithmétiques 16
4.2 Affectation composée 16
4.3 Comparer et combiner des conditions 17
4.4 Opérateur ternaire et instanceof 17

Chapitre 5 : Packages et structures de contrôle **19**
5.1 Les packages 19
5.2 Conditions : if et switch 20
5.3 Les quatre boucles 21
5.4 Instructions de rupture 22

Chapitre 6 : TD 1 : exercices **23**
6.1 Exercice 1 : facture d'électricité 23
6.2 Exercice 2 : nombres parfaits et nombres amicaux 23
6.3 Exercice 3 : changements de base 24
6.4 Exercice 4 : calendrier perpétuel 24
6.5 Exercice 5 : tableau d'amortissement d'un prêt 24
6.6 Exercice 6 : facteurs premiers, PGCD et conjecture de Goldbach 25
6.7 Exercice 7 : le jeu des allumettes contre l'ordinateur 25
6.8 Exercice 8 : relevé de températures 25
6.9 Exercice 9 : approximations numériques 26
6.10 Exercice 10 : la suite de Syracuse et ses records 26

Chapitre 7 : Tableaux et chaînes de caractères **27**
7.1 Déclarer, créer, initialiser un tableau 27
7.2 Parcourir un tableau 27
7.3 Les chaînes de caractères : String 28

Chapitre 8 : Programmation orientée objet **31**
8.1 Les quatre piliers de la POO 31
8.2 Classe et objet 31
8.3 Les constructeurs 32
8.4 Application : Marque et Téléphone 33
8.5 Encapsulation : protéger l'état 33
8.6 L'héritage : spécialiser une classe 34

8.7 Le polymorphisme 36
8.8 Surcharge ou redéfinition ? 37
8.9 Abstraction : classes abstraites et interfaces 37

Chapitre 9 : TD 2 : exercices 39
9.1 Exercice 1 : bulletins de notes 39
9.2 Exercice 2 : gestion bancaire 39
9.3 Exercice 3 : calcul matriciel 40
9.4 Exercice 4 : agence de location de véhicules 40
9.5 Exercice 5 : analyse de texte et cryptographie 41
9.6 Exercice 6 : morpion contre l'ordinateur 41
9.7 Exercice 7 : gestion de commandes et facturation 42
9.8 Exercice 8 : simulation de file d'attente 42
9.9 Exercice 9 : bataille navale 42
9.10 Exercice 10 : paie et organigramme d'entreprise 43

Annexe A : Corrigés des TD 44
A.1 TD 1 · Exercice 1 : facture d'électricité 44
A.2 TD 1 · Exercice 2 : nombres parfaits et nombres amicaux 45
A.3 TD 1 · Exercice 3 : changements de base 47
A.4 TD 1 · Exercice 4 : calendrier perpétuel 49
A.5 TD 1 · Exercice 5 : tableau d'amortissement d'un prêt 51
A.6 TD 1 · Exercice 6 : facteurs premiers, PGCD et Goldbach 52
A.7 TD 1 · Exercice 7 : le jeu des allumettes 54
A.8 TD 1 · Exercice 8 : relevé de températures 56
A.9 TD 1 · Exercice 9 : approximations numériques 57
A.10 TD 1 · Exercice 10 : la suite de Syracuse 59
A.11 TD 2 · Exercice 1 : bulletins de notes 60
A.12 TD 2 · Exercice 2 : gestion bancaire 63
A.13 TD 2 · Exercice 3 : calcul matriciel 66
A.14 TD 2 · Exercice 4 : agence de location de véhicules 68
A.15 TD 2 · Exercice 5 : analyse de texte et cryptographie 71
A.16 TD 2 · Exercice 6 : morpion contre l'ordinateur 74
A.17 TD 2 · Exercice 7 : gestion de commandes 77
A.18 TD 2 · Exercice 8 : simulation de file d'attente 80
A.19 TD 2 · Exercice 9 : bataille navale 82
A.20 TD 2 · Exercice 10 : paie et organigramme 84

Chapitre 1

L'écosystème Java

Avant d'écrire du code, il faut comprendre comment un programme Java passe du texte que vous tapez à un programme qui s'exécute, et quels outils interviennent.

Objectifs du chapitre

- Situer Java dans l'histoire et savoir le définir précisément.
- Expliquer le rôle du compilateur, du bytecode et de la JVM.
- Distinguer JDK, JRE et JVM ; comprendre le ramasse-miettes.
- Installer un JDK, écrire, compiler et exécuter un premier programme.

1.1 Java en bref

Java naît en **1991** chez **Sun Microsystems**, dans le cadre du « projet Green » mené par **James Gosling**. Le langage s'appelle d'abord Oak ; il est rebaptisé Java et présenté au public en **1995**, la version 1.0 sortant en janvier 1996. En 2007, le code du JDK devient open source sous le nom d'**OpenJDK**. En **2010**, **Oracle rachète Sun** et devient le gardien du langage.

Depuis 2018, une nouvelle version de Java sort **tous les six mois**. Certaines versions sont dites **LTS** (*Long-Term Support*) : elles reçoivent des correctifs pendant plusieurs années. Les LTS récentes sont 11 et 17 ; c'est sur l'une d'elles que l'on apprend et que l'on travaille en entreprise.

> **À retenir**
> Java est un langage **orienté objet**, à **typage statique et fort**, compilé en **bytecode** puis exécuté par une **machine virtuelle** (la JVM). Il a été créé par Sun Microsystems, pas par Oracle.

Typage statique signifie que le type de chaque variable est connu et vérifié dès la compilation : si vous essayez de ranger du texte dans une variable déclarée comme nombre entier, le compilateur refuse avant même que le programme ne tourne. **Typage fort** signifie que Java n'effectue pas de conversions hasardeuses dans votre dos.

1.2 Du code source à l'exécution

Un programme Java passe par quatre étapes.

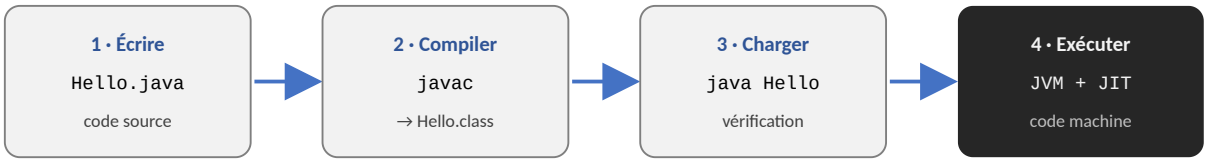


Figure 1 : Le parcours d'un programme Java

1. **Écrire** le code source dans un fichier texte d'extension `.java`, par exemple `Hello.java`.
2. **Compiler** avec `javac`, le compilateur fourni avec le JDK. Il vérifie la syntaxe et les types, puis produit un fichier `Hello.class` contenant du **bytecode**. Si le code contient une erreur, aucun fichier n'est produit et le compilateur indique la ligne fautive.
3. **Charger** : la commande `java Hello` démarre la JVM, qui charge la classe en mémoire et vérifie que le bytecode est sûr.

4. **Exécuter** : la JVM interprète le bytecode et, pour les parties exécutées souvent, le compile à la volée en code machine natif grâce à son compilateur **JIT** (*Just-In-Time*). C'est ce qui rend Java rapide malgré la machine virtuelle.

> **Pour aller plus loin**
> Le bytecode est un code intermédiaire : ce n'est ni du texte lisible ni du code machine. On peut l'afficher avec le désassembleur du JDK : `javap -c Hello`.

1.3 La JVM : « Write once, run anywhere »

Le slogan de Java signifie « écrire une fois, exécuter partout ». Le fichier `.class` produit par le compilateur est **le même** quel que soit le système. Ce qui change d'un système à l'autre, c'est la JVM : il existe une JVM pour Windows, une pour Linux, une pour macOS, etc. Chacune sait traduire le bytecode pour sa machine. On peut donc compiler un programme sous Windows et exécuter le même `.class` sous Linux : le résultat affiché est identique.

La JVM ne se contente pas d'exécuter le bytecode. Elle a quatre rôles :

Rôle	Ce que fait la JVM
Charger	Elle charge les classes en mémoire au moment où le programme en a besoin (le *class loader*).
Vérifier	Elle contrôle que le bytecode respecte les règles du langage et ne fait rien de dangereux.
Exécuter	Elle interprète le bytecode, puis compile en code natif les parties les plus utilisées (JIT).
Gérer	Elle gère la mémoire (ramasse-miettes, voir 1.5) et les threads (exécutions parallèles).

1.4 JDK, JRE, JVM : qui contient quoi ?

Ces trois sigles s'emboîtent comme des poupées russes.

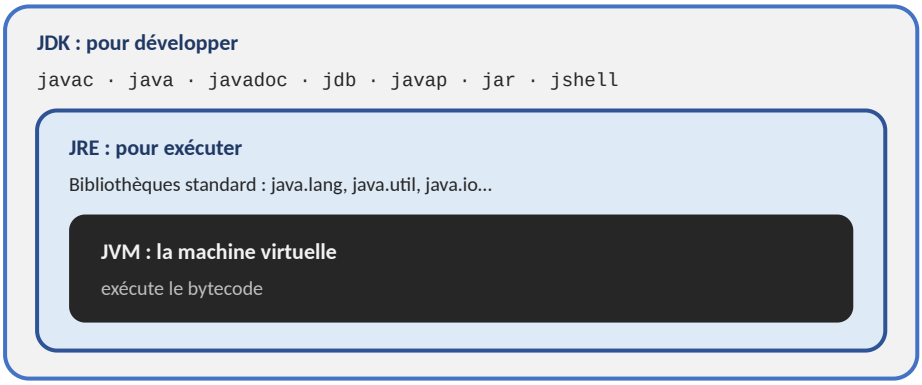


Figure 2 : Le JDK contient le JRE, qui contient la JVM

- **JVM** (*Java Virtual Machine*) : la machine virtuelle qui exécute le bytecode.
- **JRE** (*Java Runtime Environment*) = JVM + bibliothèques standard. C'est tout ce qu'il faut pour **exécuter** un programme.
- **JDK** (*Java Development Kit*) = JRE + outils de développement : le compilateur `javac`, le générateur de documentation `javadoc`, le débogueur `jdb`, le désassembleur `javap`, l'outil d'archivage `jar`, la console interactive `jshell`. C'est ce qu'il faut pour **développer**.

À retenir
Pour programmer, on installe **le JDK**. Depuis Java 11, Oracle ne distribue d'ailleurs plus de JRE séparé : le JDK suffit pour développer et pour exécuter.

Le classpath

Le **classpath** est la liste des dossiers (et des archives `.jar`) dans lesquels le compilateur et la JVM cherchent les fichiers `.class`. Par défaut, c'est le dossier courant. On peut l'indiquer avec l'option `-cp` :

```
java -cp bin Hello     # cherche Hello.class dans le dossier bin
```

1.5 Le ramasse-miettes (Garbage Collector)

En C, le programmeur doit libérer lui-même la mémoire qu'il a réservée ; un oubli provoque une « fuite mémoire », une double libération fait planter le programme. En Java, cette tâche est automatique : c'est le rôle du **ramasse-miettes**, en anglais *Garbage Collector* (GC).

La JVM utilise principalement une approche de **traçage des références** (*tracing GC*). Elle part des **racines** (*GC Roots*), c'est-à-dire les variables locales des méthodes en cours, les champs `static` et les threads actifs, puis parcourt, de référence en référence, tous les objets qu'elle peut atteindre.

- Un objet **accessible** depuis une racine **reste en mémoire**.
- Un objet **inaccessible** depuis toutes les racines est considéré comme un déchet (*garbage*) : la mémoire qu'il occupe **peut être récupérée**.

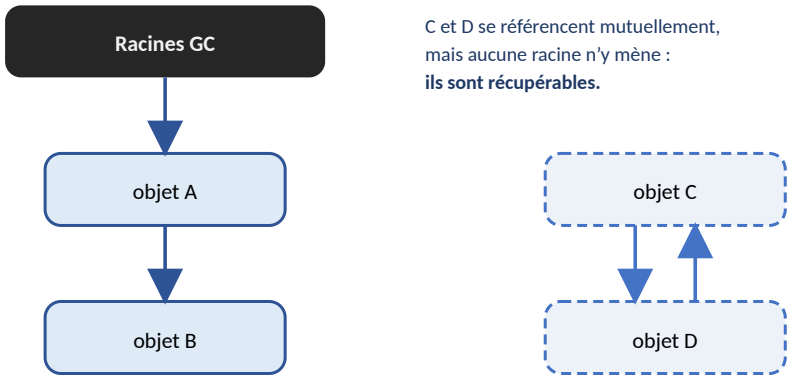


Figure 3 : A et B sont accessibles depuis les racines ; C et D ne le sont pas

Ce mécanisme gère même les **cycles** : sur la figure, C pointe vers D et D vers C. Si l'on se contentait de compter le nombre de références vers chaque objet, ni C ni D ne tomberait jamais à zéro. Le traçage, lui, constate simplement qu'aucune racine n'y mène.

À retenir
Le GC ne se contente pas de compter les références : il détermine surtout **l'accessibilité** des objets. En Java, on ne libère jamais un objet à la main ; on cesse simplement de le référencer. Le moment exact de la récupération n'est pas garanti, et `System.gc()` n'est qu'une suggestion faite à la JVM.

1.6 Installer le JDK

1. **Télécharger** : rendez-vous sur **adoptium.net**, choisissez **Eclipse Temurin**, une version LTS (11 ou 17) et votre système d'exploitation.
2. **Installer** : sous Windows, cochez dans l'installeur les options « Set JAVA_HOME variable » et « Add to PATH ». Sous macOS et Linux, l'installeur ou le gestionnaire de paquets s'en charge.
3. **Vérifier** : ouvrez un terminal (Invite de commandes, PowerShell ou Terminal) et tapez :

   ```
   java -version
   javac -version
   ```

   Les deux commandes doivent afficher le même numéro de version.
4. **Choisir un éditeur** : IntelliJ IDEA (édition Community, gratuite), Eclipse IDE, ou VS Code avec l'extension « Extension Pack for Java ».

> **Piège**
>
> - `eclipse.org/downloa`d mène à l'IDE Eclipse, pas au JDK. Le JDK Temurin se télécharge sur **adoptium.net**.
> - Si le terminal répond « javac n'est pas reconnu… », c'est presque toujours que le JDK n'a pas été ajouté au **PATH**. Relancez l'installeur en cochant l'option, puis ouvrez un *nouveau* terminal.

D'autres distributions gratuites du JDK existent et conviennent tout autant : Oracle JDK, Amazon Corretto, Microsoft Build of OpenJDK.

1.7 Votre premier programme

Créez un fichier nommé exactement `Bonjour.java` et tapez :

```
// Fichier : Bonjour.java
public class Bonjour {
    public static void main(String[] args) {
        System.out.println("Bonjour, Java !");
    }
}
```

Puis, dans un terminal ouvert dans le même dossier :

```
$ javac Bonjour.java     # compile : crée Bonjour.class
$ java Bonjour           # exécute (sans l'extension .class)
Bonjour, Java !
```

Lecture ligne par ligne

Code	Explication
`public class Bonjour`	Déclare une classe nommée `Bonjour`. En Java, **tout le code vit dans une classe**. Une classe `public` doit porter **exactement** le nom de son fichier (majuscules comprises).
`{ … }`	Les accolades délimitent un bloc : ici le contenu de la classe, puis celui de la méthode.
`public static void main(String[] args)`	La méthode `main` est le **point d'entrée** : c'est elle que la JVM appelle au démarrage. `void` signifie qu'elle ne renvoie rien ; `args` contient les éventuels arguments passés en ligne de commande.
`System.out.println(…)`	Affiche le texte entre parenthèses, puis passe à la ligne. `System.out.print` affiche sans passer à la ligne.
`;`	Chaque instruction se termine par un point-virgule.

Règles et conventions de base

- **Noms de classes** : majuscule initiale, puis majuscule à chaque mot (`CompteBancaire`). C'est une *convention* partagée par tous les développeurs Java, pas une règle du compilateur, mais on la respecte toujours.
- **Noms de variables et de méthodes** : minuscule initiale, puis majuscule à chaque mot (`nombreEleves`, `calculerMoyenne`). C'est le *camelCase*.
- **Déclarer une variable** : `type nom = valeur;` par exemple `int age = 20;`
- **Déclarer une méthode** : on précise son type de retour (`void` si elle ne renvoie rien), son nom, puis ses paramètres typés : `static int carre(int x) { return x * x; }`
- **Commentaires** : `// sur une ligne` ou `/* sur plusieurs lignes */`. Ils sont ignorés par le compilateur.
- Java est **sensible à la casse** : `System` et `system` sont deux mots différents.

Piège : les erreurs du premier programme

- Fichier nommé `bonjour.java` alors que la classe s'appelle `Bonjour` → erreur de compilation.
- Point-virgule oublié → `error: ';' expected`, avec le numéro de ligne.
- `java Bonjour.class` au lieu de `java Bonjour` → la JVM ne trouve pas la classe.

Testez-vous : Chapitre 1

1. Que contient un fichier `.class` ? Qui le produit ?
2. Pourquoi peut-on exécuter le même `.class` sous Windows et sous Linux ?
3. Quelle différence y a-t-il entre le JDK et le JRE ? Lequel faut-il installer pour programmer ?
4. Deux objets se référencent mutuellement, mais aucune variable du programme ne pointe vers eux. Seront-ils récupérés par le GC ? Pourquoi ?
5. Votre classe s'appelle `Calcul`. Comment doit s'appeler le fichier ? Quelles commandes tapez-vous pour la compiler puis l'exécuter ?

Chapitre 2

Ce qui fait la force de Java

Six caractéristiques expliquent pourquoi Java est l'un des langages les plus utilisés au monde, des applications bancaires aux applications Android.

2.1 Les six caractéristiques

Indépendant de la plateforme

Le compilateur produit du bytecode, pas du code machine. Ce bytecode tourne sur n'importe quel système disposant d'une JVM : Windows, Linux, macOS. On compile une fois, on exécute partout (voir 1.3).

Orienté objet

Un programme Java est organisé en **classes** et en **objets** qui regroupent des données et les opérations sur ces données. La programmation orientée objet repose sur quatre piliers : **abstraction**, **encapsulation**, **héritage** et **polymorphisme**. Ils sont détaillés au chapitre 8.

Plus simple que C++

Java a été conçu en retirant de C++ les fonctionnalités les plus sources d'erreurs :

- **pas de pointeurs manipulables** : on ne fait pas d'arithmétique sur les adresses mémoire ;
- **pas de surcharge d'opérateurs** : + ne peut pas être redéfini pour vos propres classes ;
- **pas d'héritage multiple de classes** : une classe n'hérite que d'une seule classe (mais elle peut implémenter plusieurs interfaces, voir 8.9) ;
- **pas de libération manuelle de la mémoire** : le ramasse-miettes s'en charge.

> **Piège**
> Ne dites pas que Java n'a « pas d'allocation explicite de la mémoire » : on alloue bien explicitement un objet avec `new`. Ce qui est automatique, c'est la **libération**.

Multithread

Un programme Java peut exécuter plusieurs **fils d'exécution** (*threads*) en parallèle, par exemple pour télécharger un fichier pendant que l'interface reste réactive, ou pour exploiter tous les cœurs du processeur.

> **Nuance**
> On entend souvent « JavaScript est monothread, Java est multithread ». C'est approximatif : JavaScript exécute votre code sur un thread principal avec une boucle d'événements, mais il dispose aussi de *workers*. La vraie différence est que Java expose directement des threads qui partagent la même mémoire.

Robuste et sûr

Le typage statique détecte beaucoup d'erreurs dès la compilation ; les **exceptions** obligent à traiter les situations anormales ; la JVM vérifie le bytecode avant de l'exécuter ; la gestion automatique de la mémoire supprime toute une famille de bugs.

Portable

La taille des types est fixée par la norme du langage : un `int` fait toujours 32 bits, quel que soit l'ordinateur. En C, elle peut varier d'une machine à l'autre. Avec l'indépendance vis-à-vis de la plateforme, c'est ce qui rend Java portable.

Testez-vous : Chapitre 2

1. Citez les quatre piliers de la programmation orientée objet.
2. Java permet-il l'héritage multiple ? Nuancez votre réponse.
3. Qu'est-ce qui est automatique en Java : l'allocation ou la libération de la mémoire ?

Chapitre 3

Types, variables et modificateurs

Une variable est une case mémoire nommée qui contient une valeur d'un type donné. Ce chapitre présente les types de Java, les différentes sortes de variables et les mots-clés qui règlent leur visibilité.

Objectifs du chapitre

- Connaître les 8 types primitifs et savoir que `String` est une classe.
- Distinguer une valeur primitive d'une référence vers un objet.
- Distinguer variables locales, d'instance et de classe (`static`).
- Savoir qui peut accéder à quoi : `private`, `protected`, `public`, niveau par défaut.

3.1 Les 8 types primitifs

Type	Taille	Plage / rôle	Exemple
`byte`	8 bits	−128 à 127	`byte b = 10;`
`short`	16 bits	−32 768 à 32 767	`short s = 1000;`
`int`	32 bits	environ ±2,1 milliards (-2^{31} à $2^{31}-1$)	`int n = 42;`
`long`	64 bits	environ $\pm 9{,}2 \times 10^{18}$	`long l = 9_000_000_000L;`
`float`	32 bits	décimal, ≈ 7 chiffres significatifs	`float f = 3.14f;`
`double`	64 bits	décimal, ≈ 15 chiffres significatifs	`double d = 3.14;`
`char`	16 bits	un caractère Unicode (UTF-16)	`char c = 'A';`
`boolean`	non précisée	`true` ou `false`	`boolean ok = true;`

En pratique, `int` et `double` couvrent la grande majorité des besoins ; `long` sert aux très grands nombres (identifiants, durées en millisecondes).

Les littéraux

- Un nombre décimal écrit dans le code (`3.14`) est un `double` par défaut. Pour un `float`, on ajoute `f` : `3.14f`.
- Un entier trop grand pour un `int` doit porter le suffixe `L` : `9000000000L`.
- Le tiret bas est autorisé pour la lisibilité : `1_000_000`.
- Un caractère s'écrit entre apostrophes droites (`'A'`), une chaîne entre guillemets droits (`"Bonjour"`).

> **Piège**
> `String` (avec un **S majuscule**) n'est **pas** un type primitif : c'est une **classe** de la bibliothèque standard. Une variable de type `String` contient une référence vers un objet (voir 3.2). Écrire `string` en minuscules provoque une erreur de compilation.

Conversions entre types

Java convertit automatiquement vers un type « plus large » (sans perte possible) : un `int` se range sans problème dans un `long` ou un `double`. Dans l'autre sens, il faut une conversion explicite, appelée **cast**, car on peut perdre de l'information :

```
int i = 42;
double d = i;          // automatique : d vaut 42.0
double x = 3.99;
int j = (int) x;       // cast explicite : j vaut 3 (la partie décimale est
tronquée)
```

Pour aller plus loin

`double` ne représente pas exactement tous les décimaux : `0.1 + 0.2` donne `0.30000000000000004`. Pour des montants d'argent, on utilise la classe `BigDecimal`.

3.2 Valeur ou référence ?

C'est une notion essentielle pour comprendre la suite du cours. Java distingue deux familles de types :

- les **types primitifs** : la variable contient directement la valeur. `int age = 20;` range le nombre 20 dans la case `age` ;
- les **types objets** (toutes les classes : `String`, `Scanner`, vos propres classes, les tableaux) : l'objet est créé dans une zone mémoire appelée le **tas** (*heap*), et la variable ne contient qu'une **référence**, c'est-à-dire l'adresse qui permet de le retrouver.

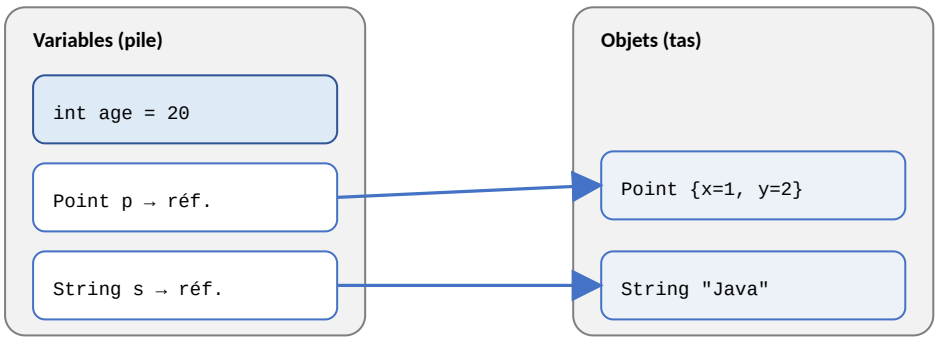


Figure 4 : Un primitif contient sa valeur ; une variable objet contient une référence

Conséquence directe : l'opérateur == compare le contenu des variables. Pour deux primitifs, il compare les valeurs ; pour deux objets, il compare **les références**, c'est-à-dire « est-ce le même objet ? ». Pour comparer le **contenu** de deux objets, on utilise la méthode `equals()`.

```
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b);        // false : deux objets distincts
System.out.println(a.equals(b));   // true  : même contenu
```

Autre conséquence : si l'on écrit `Point q = p;`, on ne copie pas le point ; on copie la référence. `p` et `q` désignent alors **le même objet**, et une modification faite via `q` est visible via `p`.

À retenir

== compare des références ; `equals()` compare des contenus. Pour comparer deux chaînes, on écrit toujours `s1.equals(s2)`.

3.3 Trois sortes de variables

Sorte	Où et comment	Exemple
Locale	Déclarée dans une méthode, un constructeur ou un bloc. N'existe que dans ce bloc. **Doit être initialisée** avant d'être utilisée, sinon erreur de compilation.	`void f() { int x = 5; }`
D'instance (attribut)	Déclarée dans la classe, hors de toute méthode. **Chaque objet possède sa propre copie.** Reçoit une valeur par défaut : `0`, `false` ou `null`.	`class Chat { String nom; }`
De classe (`static`)	Déclarée dans la classe avec le mot-clé `static`. **Une seule copie**, partagée par tous les objets. Existe dès le chargement de la classe.	`static int compteur = 0;`

Deux notions à ne pas confondre :

- la **portée** : la zone du code où le nom de la variable est utilisable ;
- la **durée de vie** : le temps pendant lequel la valeur existe en mémoire.

```
public class Chat {
    static int nombreDeChats = 0;     // de classe : partagée
    String nom;                       // d'instance : une par chat

    Chat(String nom) {
        this.nom = nom;
        nombreDeChats++;              // compte les objets créés
    }

    void miauler() {
        String cri = "Miaou";         // locale : n'existe que dans miauler()
        System.out.println(nom + " : " + cri);
    }
}
```

3.4 Variable static ou variable d'instance ?

Critère	Variable `static` (de classe)	Variable d'instance
Copies	une seule, partagée	une par objet
Création	au chargement de la classe par la JVM	à chaque `new`
Disparition	au déchargement de la classe (en pratique, à la fin du programme)	quand le GC récupère l'objet devenu inaccessible
Accès	`NomClasse.variable`	`objet.variable` (ou `this.variable` dans la classe)
Exemple	`Chat.nombreDeChats`	`monChat.nom`

À retenir

Une variable d'instance s'atteint toujours **par un objet**, jamais par le nom de la classe. Seule une variable `static` s'écrit `NomClasse.variable`.

Le mot-clé `this`

Dans une méthode ou un constructeur, `this` désigne **l'objet courant**, celui sur lequel la méthode a été appelée. On l'utilise surtout quand un paramètre porte le même nom qu'un attribut : `this.nom = nom;` signifie « l'attribut `nom` de cet objet reçoit la valeur du paramètre `nom` ».

3.5 Exercice : trouvez les trois erreurs

Énoncé
Le code suivant contient trois erreurs. Trouvez-les avant de lire la correction.

```
class Toto() {
    static int nbr = 0;
    Toto val1 = new Toto();
    void set(int v) {
        this.nbr = v;
    }
}
```

Correction

1. **`class Toto()`** : une déclaration de classe ne prend pas de parenthèses. On écrit `class Toto { … }`.
2. **`Toto val1 = new Toto();`** comme attribut : chaque fois qu'on crée un `Toto`, son attribut `val1` crée un autre `Toto`, qui en crée un autre… à l'infini. Le programme s'arrête avec une `StackOverflowError`.
3. **`this.nbr`** : `this` désigne l'objet courant et sert aux attributs d'instance. `nbr` est `static` : on écrit `Toto.nbr`. Le code compile avec un avertissement, mais il est trompeur.

```
public class Toto {
    int jour;                  // une copie par objet
    static int nbr = 0;        // partagée par tous les objets

    Toto(int jour) {
        this.jour = jour;      // attribut de l'objet courant
        nbr++;                 // ou Toto.nbr++
    }
}
```

3.6 Modificateurs : accès et comportement

Les **modificateurs d'accès** indiquent quelles parties du programme peuvent utiliser un attribut, une méthode ou une classe.

Mot-clé	Classe	Package	Sous-classe (autre package)	Partout
`private`	oui	non	non	non
(aucun mot-clé)	oui	oui	non	non
`protected`	oui	oui	oui	non
`public`	oui	oui	oui	oui

Piège

- `protected` ne signifie pas « visible seulement dans le package » : il ouvre l'accès au package **et** aux sous-classes, même situées dans un autre package.
- Sans aucun mot-clé, le membre est visible **dans le package seulement** : c'est le niveau par défaut, dit *package-private*.

Deux autres mots-clés modifient le **comportement**, pas la visibilité :

- `static` : le membre appartient à la classe, et non à chaque objet ;
- `final` : une variable `final` ne peut être affectée qu'une fois (c'est une constante) ; une méthode `final` ne peut pas être redéfinie ; une classe `final` ne peut pas avoir de sous-classe.

```
public static final double TAUX_TVA = 0.18;   // constante : nom en MAJUSCULES
```

À retenir

Règle pratique : attributs `private`, méthodes utiles `public`, et ouvrir le moins possible.

Testez-vous : Chapitre 3

1. Quel type choisir pour stocker un prix avec centimes ? Et une réponse oui/non ?
2. Que vaut `(int) 7.8` ?
3. Pourquoi `s1 == s2` peut-il renvoyer `false` pour deux chaînes qui contiennent le même texte ?
4. Une classe `Eleve` possède un attribut `static int effectif`. Si l'on crée trois élèves, combien de copies de `effectif` existent ?
5. Un attribut `protected` est-il visible depuis une sous-classe située dans un autre package ?

Chapitre 4

Les opérateurs

Les opérateurs permettent de calculer, d'affecter, de comparer et de combiner des conditions.

4.1 Opérateurs arithmétiques

Opérateur	Rôle	Exemple
`+`	addition	`7 + 2` → `9`
`-`	soustraction	`7 - 2` → `5`
`*`	multiplication	`7 * 2` → `14`
`/`	division	`7 / 2` → `3` (division entière !) ; `7.0 / 2` → `3.5`
`%`	modulo : reste de la division entière	`7 % 2` → `1`
`++` / `--`	ajoute / retire 1	`i++` équivaut à `i = i + 1`

Piège : la division entière
Entre deux entiers, `/` donne un résultat entier : la partie décimale est **tronquée**. `7 / 2` vaut `3`, pas `3.5`. Pour obtenir 3.5, au moins un des deux opérandes doit être décimal : `7 / 2.0` ou `(double) 7 / 2`. Et la division d'un entier par zéro provoque une erreur à l'exécution : `ArithmeticException`.

Le modulo est très utile : `n % 2 == 0` teste si n est pair ; `n % 10` donne le dernier chiffre de `n`.

Priorité des opérateurs

Comme en mathématiques, `*`, `/` et `%` sont évalués avant `+` et `-`. Les parenthèses permettent de forcer l'ordre et lèvent toute ambiguïté : `2 + 3 * 4` vaut `14`, `(2 + 3) * 4` vaut `20`.

Le + qui concatène

Dès qu'un des opérandes est une chaîne, `+` concatène (met bout à bout). L'évaluation se fait de gauche à droite :

```
System.out.println("a" + 1 + 2);    // "a12" : "a"+1 donne "a1", puis "a1"+2
System.out.println(1 + 2 + "a");    // "3a"  : 1+2 vaut 3, puis 3+"a"
System.out.println("a" + (1 + 2));  // "a3"  : les parenthèses d'abord
```

4.2 Affectation composée

Une seule règle : **`x op= y` équivaut à `x = x op y`**. Exemples avec `x = 10` au départ :

Écriture	Équivaut à	Résultat
`x += 3`	`x = x + 3`	13
`x -= 3`	`x = x - 3`	7
`x *= 3`	`x = x * 3`	30
`x /= 3`	`x = x / 3`	3 (division entière)
`x %= 3`	`x = x % 3`	1

Piège
Attention à l'ordre : `x -= 3` retire 3 **à x**. Ce n'est pas « 3 moins x ». De même, `x /= 3` divise x par 3.

Pour aller plus loin
L'opérateur composé inclut une conversion automatique. Avec `byte b = 10;`, l'instruction `b += 5;` compile, alors que `b = b + 5;` est refusée : `b + 5` est calculé en `int`, qu'on ne peut pas ranger dans un `byte` sans cast.

4.3 Comparer et combiner des conditions

Opérateurs relationnels

Ils renvoient un `boolean`.

Op.	Sens	`5 op 3`
`==`	égal à	false
`!=`	différent de	true
`<`	inférieur	false
`<=`	inférieur ou égal	false
`>`	supérieur	true
`>=`	supérieur ou égal	true

Opérateurs logiques

Ils combinent des conditions.

`a`	`b`	`a && b`	`a || b`	`!a`
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

`&&` : ET ; `||` : OU ; `!` : NON.

Le court-circuit

`&&` et `||` s'arrêtent dès que le résultat est connu. Dans `a && b`, si a est faux, `b` n'est même pas évalué. On s'en sert pour éviter des erreurs :

```
if (nom != null && nom.length() > 0) {   // length() n'est appelé que si nom n'est pas null
    System.out.println("Bonjour " + nom);
}
```

Pièges

- Ne confondez pas = (affectation) et == (comparaison).
- « Hors de l'intervalle [0 ; 1000] » s'écrit `n < 0 || n > 1000` (OU). La condition `n < 0 && n > 1000` ne peut jamais être vraie.
- Sur des objets, == compare les références : pour deux `String`, utilisez `equals()` (voir 3.2).

4.4 Opérateur ternaire et instanceof

L'opérateur ternaire `condition ? siVrai : siFaux`

C'est une **expression** qui produit une valeur : si la condition est vraie, elle vaut la première valeur, sinon la seconde. On l'utilise pour une affectation ou un affichage simple.

```
int age = 20;
String statut = (age >= 18) ? "majeur" : "mineur";   // "majeur"
```

Au-delà d'une condition simple, préférez un `if/else`, plus lisible.

L'opérateur `instanceof`

`objet instanceof Type` renvoie `true` si l'objet est une instance de ce type, d'une de ses sous-classes ou d'une classe qui implémente cette interface. Il renvoie `false` si l'objet vaut `null`. Depuis Java 16, il peut déclarer au passage une variable du bon type :

```
Object o = "Bonjour";
if (o instanceof String s) {
    System.out.println(s.length());   // 7 : s est déjà de type String
}
```

Testez-vous : Chapitre 4

1. Que valent `17 / 5`, `17 % 5` et `17.0 / 5` ?
2. Qu'affiche `System.out.println("Total : " + 2 + 3);` ? Comment obtenir « Total : 5 » ?
3. Après `int x = 20; x -= 5; x /= 2;`, que vaut `x` ?
4. Écrivez la condition « n est compris entre 1 et 10 inclus ».
5. Réécrivez avec un ternaire : `if (n % 2 == 0) parite = "pair"; else parite = "impair";`

Chapitre 5

Packages et structures de contrôle

Un programme n'exécute pas toujours ses instructions dans l'ordre : il prend des décisions et répète des actions. Ce chapitre présente aussi l'organisation du code en packages et la lecture au clavier.

5.1 Les packages

Un **package** regroupe des classes, des interfaces et des sous-packages dans un même espace de noms. Il correspond à un dossier sur le disque. Les packages évitent les conflits de noms et organisent les grands projets.

Les packages de la bibliothèque standard

Package	Contenu
`java.lang`	`String`, `Math`, `System`, `Integer`… **Importé automatiquement** : pas besoin d'`import`.
`java.util`	Collections (`ArrayList`, `HashMap`), `Scanner`, `Random`, `Arrays`.
`java.io`, `java.nio`	Lecture et écriture de fichiers, flux d'entrée-sortie.
`java.time`	Dates et heures : `LocalDate`, `LocalDateTime`, `Duration`.
`java.net`	Réseau : URI, sockets, client HTTP.
`javax.swing`	Interfaces graphiques classiques.

Déclarer et importer

```
package fr.ecole.td1;            // 1re instruction du fichier : le package de la classe

import java.util.Scanner;        // importe une classe
import java.util.*;              // importe toutes les classes du package
```

Sans `import`, on peut toujours écrire le nom complet : `java.util.Scanner sc = new java.util.Scanner(System.in);`. Une classe sans instruction `package` est placée dans le package « sans nom » : pratique pour un petit test, à éviter dans un vrai projet.

> **À retenir**
> Les noms de packages s'écrivent **en minuscules**, souvent à partir d'un nom de domaine inversé : `fr.ecole.td1`. Pour les dates, utilisez `java.time` plutôt que l'ancienne classe `java.util.Date`.

Lire au clavier avec `Scanner`

Les exercices du TD demandent de saisir des valeurs. On utilise pour cela la classe `Scanner` du package `java.util` :

```java
import java.util.Scanner;

public class Saisie {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);     // lit ce qui est tapé au clavier
        System.out.print("Votre âge : ");
        int age = sc.nextInt();                  // lit un entier
        System.out.print("Votre taille (m) : ");
        double taille = sc.nextDouble();         // lit un décimal
        sc.nextLine();                           // consomme le retour à la ligne
restant
        System.out.print("Votre nom : ");
        String nom = sc.nextLine();              // lit toute une ligne de texte
        System.out.println(nom + ", " + age + " ans, " + taille + " m");
        sc.close();
    }
}
```

Piège
Après `nextInt()` ou `nextDouble()`, le retour à la ligne tapé par l'utilisateur reste en attente. Un `nextLine()` appelé juste après renverrait une chaîne vide : c'est pourquoi on ajoute un `sc.nextLine();` « vide » avant de lire du texte. Attention aussi : selon la langue de l'ordinateur, un décimal se saisit avec une virgule (`1,75`) ou un point.

5.2 Conditions : if et switch

`if`, `else if`, `else`

```java
int note = 14;
if (note >= 16) {
    System.out.println("Très bien");
} else if (note >= 10) {
    System.out.println("Admis");
} else {
    System.out.println("Ajourné");
}
```

Les conditions sont testées **dans l'ordre** ; seul le premier bloc dont la condition est vraie s'exécute. On teste donc du plus restrictif au moins restrictif : si l'on testait `note >= 10` en premier, un 18 afficherait « Admis ».

Piège
`if` n'est **pas une boucle** : c'est une structure conditionnelle, exécutée au plus une fois. Elle ne répète rien.

`switch`

Quand on compare une même variable à plusieurs valeurs précises, `switch` est plus lisible. La forme moderne (Java 14 et plus) utilise des flèches, n'a pas besoin de `break` et peut produire une valeur :

```
String jour = "SAMEDI";
String type = switch (jour) {
    case "SAMEDI", "DIMANCHE" -> "week-end";
    default -> "semaine";
};
```

L'ancienne forme, que vous rencontrerez dans du code existant, utilise `case …:` et `break`. Oublier un `break` y provoque l'exécution des cas suivants « en cascade ».

```
switch (choix) {
    case 1:
        System.out.println("Nouvelle partie");
        break;
    case 2:
        System.out.println("Charger");
        break;
    default:
        System.out.println("Choix invalide");
}
```

5.3 Les quatre boucles

Boucle	Quand l'utiliser	Exemple
`for`	Le nombre d'itérations est connu à l'avance.	`for (int i = 1; i <= 5; i++) { … }`
`while`	Tant qu'une condition est vraie (testée **avant** chaque tour).	`while (solde < objectif) { … }`
`do … while`	Au moins une exécution (test **après**) : idéal pour redemander une saisie.	`do { … } while (n < 0);`
for-each	Parcourir tous les éléments d'un tableau ou d'une collection.	`for (int note : notes) { … }`

La boucle `for` en détail

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);        // affiche 1, 2, 3, 4, 5
}
```

L'en-tête contient trois parties séparées par des points-virgules : l'**initialisation** (`int i = 1`, exécutée une fois), la **condition** (`i <= 5`, testée avant chaque tour) et la **mise à jour** (`i++`, exécutée après chaque tour).

`while` et `do … while`

```
double solde = 100;
int annees = 0;
while (solde < 200) {             // combien d'années pour doubler à 5 % ?
    solde *= 1.05;
    annees++;
}
System.out.println(annees + " ans");   // 15 ans

int n;
do {
    System.out.print("Entier positif : ");
    n = sc.nextInt();
} while (n < 0);                 // on redemande tant que la saisie est invalide
```

Piège : la boucle infinie

Si la condition ne devient jamais fausse, la boucle ne s'arrête pas. Vérifiez toujours que la variable testée évolue à chaque tour dans la bonne direction.

5.4 Instructions de rupture

- `break` : sort immédiatement de la boucle (ou du `switch`).
- `continue` : abandonne le tour en cours et passe directement au tour suivant.
- `return` : quitte **toute la méthode**, pas seulement la boucle, en renvoyant une valeur si la méthode n'est pas `void`.

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;    // saute les nombres pairs
    if (i > 7) break;            // arrête tout au-delà de 7
    System.out.print(i + " ");
}
// Affiche : 1 3 5 7
```

Tracez cette boucle à la main, tour par tour, en notant la valeur de `i` et ce qui se passe : c'est le meilleur moyen de comprendre `break` et `continue`.

Testez-vous : Chapitre 5

1. Faut-il importer la classe `Math` ? Pourquoi ?
2. Une note de 18 est testée par `if (note >= 10) … else if (note >= 16) ….` Quel message s'affiche ? Corrigez.
3. Quelle boucle choisir pour redemander une saisie tant qu'elle est invalide ?
4. Écrivez une boucle qui affiche les multiples de 3 de 3 à 30.
5. Quelle différence y a-t-il entre `break` et `return` ?

Chapitre 6

TD 1 : exercices

Dix exercices de synthèse sur les chapitres 1 à 5 : types, opérateurs, conditions, boucles, méthodes, récursivité et saisie au clavier. Les tableaux ne sont pas autorisés dans ce TD. Chaque exercice peut servir de sujet d'examen. Les corrigés commentés figurent en annexe A.

6.1 Exercice 1 : facture d'électricité

Une compagnie d'électricité facture la consommation de ses clients selon une **tarification progressive** : chaque tranche n'est payée qu'au prix de cette tranche.

Tranche	Prix du kWh
de 0 à 20 kWh (tranche sociale)	76 F
de 21 à 250 kWh	112 F
au-delà de 250 kWh	135 F

À l'énergie s'ajoutent une prime fixe de 1 000 F et une TVA de 18 %, appliquée **uniquement** au montant de l'énergie situé au-delà de la tranche sociale. Par exemple, pour 100 kWh : énergie = 20 × 76 + 80 × 112 = 10 480 F ; TVA = 18 % de 8 960 F = 1 613 F ; net à payer = 13 093 F.

1. Déclarez les prix, les bornes des tranches, la prime et le taux de TVA comme constantes.
2. Écrivez une méthode `double montantEnergie(int kwh)`, puis une méthode `double montantTaxable(int kwh)` qui renvoie la part de l'énergie soumise à la TVA. Indice : `montantTaxable` peut s'écrire à l'aide de `montantEnergie`.
3. Le programme traite les clients les uns après les autres. Pour chacun, il lit l'ancien et le nouvel index du compteur ; le nouvel index doit être supérieur ou égal à l'ancien, sinon il est redemandé. Un ancien index égal à −1 termine la saisie ; tout autre index négatif est refusé.
4. Pour chaque client, affichez la consommation, le montant de l'énergie, la TVA, la prime fixe et le net à payer.
5. À la fin, affichez le nombre de clients, le total facturé, la facture moyenne, la plus forte consommation et le pourcentage de clients restés dans la tranche sociale. Le programme ne doit pas planter si aucun client n'a été saisi.

6.2 Exercice 2 : nombres parfaits et nombres amicaux

Soit $s(n)$ la somme des diviseurs de n strictement inférieurs à n. Un entier est **parfait** si $s(n) = n$ ($6 = 1 + 2 + 3$), **abondant** si $s(n) > n$, **déficient** sinon. Deux entiers distincts a et b sont **amicaux** si $s(a) = b$ et $s(b) = a$ (220 et 284).

1. Écrivez `long sommeDiviseursPropres(int n)`. Elle ne doit tester les diviseurs que jusqu'à $\sqrt{n}$; justifiez en commentaire pourquoi c'est suffisant et comment on retrouve les autres diviseurs.
2. Écrivez `String nature(int n)` qui renvoie « parfait », « abondant » ou « déficient ».
3. Pour n de 1 à 10 000, affichez les nombres parfaits et comptez les nombres de chaque catégorie.
4. Déterminez le nombre « le plus abondant » jusqu'à 10 000, c'est-à-dire celui pour lequel le rapport $s(n) / n$ est le plus grand.
5. Affichez tous les couples de nombres amicaux inférieurs ou égaux à 10 000, chaque couple une seule fois. Expliquez comment votre programme évite les doublons et pourquoi il n'affiche pas les nombres parfaits.

6.3 Exercice 3 : changements de base

On interdit dans cet exercice les méthodes toutes faites de Java comme `Integer.toBinaryString` ou `Integer.parseInt`.

1. Écrivez `char chiffre(int valeur)` qui renvoie le caractère correspondant à une valeur de 0 à 15 (`'0'` à `'9'`, puis `'A'` à `'F'`), en utilisant l'arithmétique sur les caractères.
2. Écrivez `String versBase(int n, int base)` qui convertit un entier positif dans une base de 2 à 16 par divisions successives. Exemple : `versBase(2022, 16)` renvoie `"7E6"`.
3. On saisit un nombre binaire sous la forme d'un `long` composé de 0 et de 1, par exemple `101101`. Écrivez `long binaireVersDecimal(long binaire)` qui renvoie sa valeur décimale (45), ou `-1` si l'un des chiffres n'est ni 0 ni 1.
4. Écrivez `int bitsA1(int n)` qui compte les bits à 1 de *n*, puis `boolean estPuissanceDe2(int n)` qui l'utilise.
5. Regroupez ces fonctions dans un menu (`do … while` et `switch`) qui se répète jusqu'au choix 0.
6. Question de réflexion : pourquoi la question 3 utilise-t-elle un `long` et non un `int` ? Quel est le plus grand nombre binaire que l'on peut ainsi saisir ?

6.4 Exercice 4 : calendrier perpétuel

1. Écrivez `estBissextile`, `joursDansMois` (avec un `switch`) et `dateValide` ; seules les années à partir de 1583 (calendrier grégorien) sont acceptées.
2. La **congruence de Zeller** donne le jour de la semaine d'une date *j*/*m*/*a* : si $m < 3$, on remplace *m* par $m + 12$ et *a* par $a - 1$; on pose $k = a \bmod 100$ et $s = a / 100$; alors $h = (j + 13(m+1)/5 + k + k/4 + s/4 + 5s) \bmod 7$, avec 0 = samedi, 1 = dimanche, 2 = lundi… Écrivez `int jourSemaine(int j, int m, int a)` qui renvoie 1 pour lundi et 7 pour dimanche. Vérifiez que le 10 janvier 2022 est un lundi.
3. Écrivez `int numeroDansAnnee(int j, int m, int a)` : le 1er janvier vaut 1, le 31 décembre vaut 365 ou 366.
4. Écrivez `void afficherMois(int m, int a)` qui affiche le calendrier du mois, semaine par semaine, aligné sous l'en-tête `Lu Ma Me Je Ve Sa Di`.
5. Le programme lit deux dates valides (redemandées tant qu'elles ne le sont pas), affiche le jour de la semaine et le numéro dans l'année de la première, le calendrier de son mois, puis le nombre de jours qui séparent les deux dates, dans un ordre quelconque.

Exemple d'affichage attendu pour janvier 2022 :

```
 Lu Ma Me Je Ve Sa Di
                 1  2
  3  4  5  6  7  8  9
 10 11 12 13 14 15 16
 17 18 19 20 21 22 23
 24 25 26 27 28 29 30
 31
```

6.5 Exercice 5 : tableau d'amortissement d'un prêt

Pour un capital *C* emprunté au taux annuel *T* % sur *n* mois, la mensualité constante vaut $M = C \times t / (1 - (1+t)^{-n})$, où $t = T / 1200$ est le taux mensuel. Chaque mois, les intérêts valent $t \times$ capital restant dû ; le reste de la mensualité rembourse le capital.

1. Écrivez `double mensualite(double capital, double tauxAnnuel, int nbMois)`. Traitez le cas d'un taux nul.

2. Lisez le capital (strictement positif), le taux (entre 0 et 30 %) et la durée (de 1 à 360 mois), en redemandant chaque valeur tant qu'elle est invalide.
3. Affichez le tableau d'amortissement : mois, mensualité, intérêts, capital remboursé, capital restant dû. La dernière échéance doit solder exactement le prêt.
4. Affichez le total des intérêts et le coût total du crédit.
5. L'emprunteur indique son budget mensuel maximal. Écrivez une méthode qui renvoie la plus petite durée (au plus 360 mois) dont la mensualité tient dans ce budget, ou −1 si c'est impossible, et affichez le résultat.

Contrôle : 1 000 000 F à 12 % sur 12 mois donnent une mensualité de 88 849 F et 66 185 F d'intérêts.

6.6 Exercice 6 : facteurs premiers, PGCD et conjecture de Goldbach

1. Écrivez `boolean estPremier(int n)` efficace.
2. Écrivez `String decomposition(int n)` qui renvoie la décomposition en facteurs premiers sous la forme `"360 = 2^3 x 3^2 x 5"` : les exposants égaux à 1 ne sont pas écrits.
3. Écrivez une méthode **récursive** `long pgcd(long a, long b)` fondée sur l'algorithme d'Euclide, puis `long ppcm(long a, long b)`. Expliquez pourquoi il vaut mieux calculer `a / pgcd(a, b) * b` que `a * b / pgcd(a, b)`.
4. Le programme lit deux entiers supérieurs à 1 et affiche leurs décompositions, leur PGCD, leur PPCM, et s'ils sont premiers entre eux.
5. La **conjecture de Goldbach** affirme que tout entier pair supérieur ou égal à 4 est la somme de deux nombres premiers. Écrivez une méthode qui compte le nombre de façons d'écrire un pair n sous la forme $p + q$ ($p \le q$ premiers), vérifiez la conjecture jusqu'à une limite saisie et affichez le pair ayant le plus de décompositions.

6.7 Exercice 7 : le jeu des allumettes contre l'ordinateur

Deux joueurs retirent à tour de rôle 1, 2 ou 3 allumettes d'un tas ; celui qui prend la **dernière** allumette a perdu.

1. Analyse : en partant des petites valeurs, montrez que le joueur qui doit jouer devant 1, 5, 9, 13… allumettes perd forcément si son adversaire joue bien. Déduisez-en la stratégie gagnante.
2. Écrivez `int coupOrdinateur(int n)` qui applique cette stratégie ; lorsque l'ordinateur est lui-même en position perdante, il joue au hasard (`Random`) un coup autorisé.
3. Écrivez `int coupJoueur(Scanner sc, int n)` qui redemande la saisie tant qu'elle est invalide : jamais plus de 3, ni plus que le nombre d'allumettes restantes.
4. Écrivez `boolean partie(Scanner sc, int n, boolean joueurCommence)` qui affiche le tas (`|||||`) à chaque tour et renvoie `true` si le joueur humain gagne.
5. Le programme demande le nombre d'allumettes (de 10 à 50) et qui commence, enchaîne les parties tant que le joueur veut rejouer, et tient le score.
6. Avec 21 allumettes, qui doit commencer pour être sûr de gagner ? Justifiez.

6.8 Exercice 8 : relevé de températures

On saisit les températures de n jours consécutifs ($n \ge 3$). **Les tableaux étant interdits**, toutes les statistiques doivent être calculées au fil de la saisie, en un seul passage.

1. Calculez la moyenne et l'**écart-type** σ. Indice : σ^2 = (moyenne des carrés) − (carré de la moyenne).
2. Trouvez la température minimale et la maximale, avec le numéro du jour correspondant (en cas d'égalité, le premier jour).

3. Trouvez la plus longue série de hausses strictes consécutives (par exemple 19, 23, 26 est une série de 3 jours) et indiquez son jour de début et son jour de fin.
4. Comptez les **pics** : un jour est un pic si sa température est strictement supérieure à celles de la veille et du lendemain. Quelles variables faut-il conserver d'un tour de boucle à l'autre ?
5. Question de réflexion : peut-on, toujours sans tableau et en un seul passage, compter les jours dont la température dépasse la moyenne ? Justifiez.

Contrôle : pour 20, 22, 25, 21, 19, 23, 26, 24, on obtient une moyenne de 22,50, un écart-type de 2,29, deux pics et une plus longue hausse de 3 jours (du jour 1 au jour 3).

6.9 Exercice 9 : approximations numériques

1. **Leibniz** : $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \dots$ Écrivez une méthode qui renvoie le nombre de termes nécessaires pour approcher π à une précision donnée.
2. **Wallis** : $\pi/2 = (2/1 \times 2/3) \times (4/3 \times 4/5) \times (6/5 \times 6/7) \times \dots$ Même question.
3. Affichez un tableau comparatif pour les précisions 10^{-1} à 10^{-5}. Que constatez-vous sur la vitesse de convergence de chaque méthode ?
4. **Héron** : la suite $x_{k+1} = (x_k + a/x_k)/2$ converge vers $\sqrt{a}$. Programmez-la avec un arrêt lorsque deux termes successifs diffèrent de moins de 10^{-12}, affichez le nombre d'itérations et comparez avec `Math.sqrt` pour a = 2, 2 000 et 2 000 000.
5. **Exponentielle** : $e^x = 1 + x + x^2/2! + x^3/3! + \dots$ Programmez la série **sans calculer de factorielle** (chaque terme se déduit du précédent). Comparez avec `Math.exp` pour $x = -10$, 5 et 20. Pourquoi le résultat est-il moins précis pour $x = -10$?

6.10 Exercice 10 : la suite de Syracuse et ses records

On part d'un entier $n > 0$. S'il est pair, on le divise par 2 ; sinon, on le remplace par $3n + 1$; on s'arrête en atteignant 1. La **durée du vol** est le nombre d'étapes, l'**altitude maximale** la plus grande valeur atteinte, la **durée en altitude** le nombre d'étapes pendant lesquelles la suite reste strictement au-dessus de sa valeur de départ.

1. Écrivez `suivant`, `dureeVol`, `altitudeMax` et `dureeEnAltitude`. Les termes de la suite sont calculés en `long`.
2. Lisez un entier et affichez la suite complète sur une ligne, puis ses trois caractéristiques. Pour 27 : 111 étapes et une altitude de 9 232.
3. Un départ n est un **record** si son vol est plus long que celui de tous les départs plus petits. Affichez tous les records jusqu'à une limite saisie.
4. Trouvez le plus petit départ dont l'altitude maximale dépasse la capacité d'un `int`. Que se serait-il passé si la suite avait été calculée en `int` ?

Chapitre 7

Tableaux et chaînes de caractères

Un tableau range plusieurs valeurs du même type sous un seul nom. Une chaîne de caractères représente du texte. Ce sont deux outils que vous utiliserez dans presque tous vos programmes.

7.1 Déclarer, créer, initialiser un tableau

Un tableau se manipule en trois temps.

1. Déclarer

```
int[] notes;          // forme recommandée
int notes2[];         // forme admise, héritée du C, déconseillée
```

La déclaration crée seulement une **référence** : aucun tableau n'existe encore (la variable vaut `null`).

2. Instancier (créer)

```
notes = new int[5];           // réserve 5 cases, toutes initialisées à 0
int[] ages = new int[30];     // déclaration et création en une ligne
```

`new` réserve la mémoire. Le nombre entre crochets est la **taille** : il fixe le nombre d'éléments. Le type (`int`) fixe ce que contient chaque case.

3. Ou initialiser directement

```
int[] t = {12, 15, 9};               // la taille (3) est déduite des valeurs
int[] u = new int[]{12, 15, 9};      // forme longue, équivalente
String[] jours = {"lundi", "mardi", "mercredi"};
```

Valeurs par défaut des cases

Type des cases	Valeur par défaut
`int`, `long`, `double`…	`0` (ou `0.0`)
`boolean`	`false`
`char`	le caractère nul `'\u0000'`
objets (`String`, vos classes…)	`null`

> **Piège**
> La taille d'un tableau est **définitive** : un tableau ne peut pas grandir. Si vous avez besoin d'une liste qui s'agrandit, utilisez `ArrayList` (package `java.util`).

7.2 Parcourir un tableau

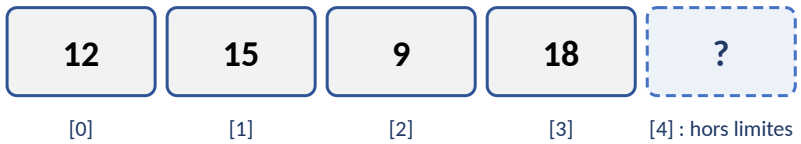


Figure 5 : Le tableau `notes = {12, 15, 9, 18}` : indices de 0 à 3

Les cases sont numérotées par un **indice** qui commence à **0**. Un tableau de taille 4 a donc des indices de 0 à 3. On accède à une case par `notes[i]` et on connaît la taille par `notes.length` (sans parenthèses : ce n'est pas une méthode).

```
int[] notes = {12, 15, 9, 18};
notes[2] = 11;                              // modifie la 3e case

// Parcours avec l'indice
for (int i = 0; i < notes.length; i++) {
    System.out.println("notes[" + i + "] = " + notes[i]);
}

// Parcours sans indice (for-each)
int somme = 0;
for (int n : notes) {
    somme += n;
}
System.out.println("Moyenne : " + (double) somme / notes.length);
```

Pièges

- Accéder à `notes[4]` dans un tableau de taille 4 provoque une `ArrayIndexOutOfBoundsException`.
- L'erreur classique « d'un en trop » : écrire `i <= notes.length` au lieu de `i < notes.length`.
- La boucle *for-each* lit chaque valeur, mais ne permet ni de modifier les cases ni de connaître l'indice.

Pour afficher un tableau d'un coup : `System.out.println(java.util.Arrays.toString(notes));` affiche `[12, 15, 11, 18]`.

Tableaux à deux dimensions

Un tableau 2D est un **tableau de tableaux** : on ajoute une paire de crochets par dimension.

```
int[][] m = new int[3][4];     // 3 lignes, 4 colonnes
m[1][2] = 7;                   // ligne 1, colonne 2

for (int i = 0; i < m.length; i++) {           // m.length : nombre de lignes
    for (int j = 0; j < m[i].length; j++) {    // m[i].length : taille de la ligne
i
        System.out.print(m[i][j] + " ");
    }
    System.out.println();
}
```

Comme chaque ligne est un tableau à part, les lignes peuvent avoir des longueurs différentes.

7.3 Les chaînes de caractères : String

```
String maChaine = "Bonjour à tous";
```

`String` est une classe. Elle offre de nombreuses méthodes :

Méthode	Rôle	Avec `s = "Bonjour"`
`s.length()`	nombre de caractères	`7`
`s.charAt(i)`	caractère à l'indice `i`	`s.charAt(0)` → `'B'`
`s.toUpperCase()`	copie en majuscules	`"BONJOUR"`
`s.substring(a, b)`	sous-chaîne de a inclus à b exclu	`s.substring(0, 3)` → `"Bon"`
`s.indexOf("jour")`	position d'une sous-chaîne (−1 si absente)	`3`
`s.contains("jour")`	contient-elle ce texte ?	`true`
`s.equals(t)`	même contenu que `t` ?	`s.equals("Bonjour")` → `true`
`s.equalsIgnoreCase(t)`	idem, sans tenir compte de la casse	`s.equalsIgnoreCase("BONJOUR")` → `true`
`s.trim()`	supprime les espaces au début et à la fin	

Une chaîne est immuable

Une fois créée, une `String` ne change plus jamais. Les méthodes comme `toUpperCase()` ne modifient pas la chaîne : elles en **renvoient une nouvelle**.

```
String s = "Bonjour";
s.toUpperCase();          // ne modifie PAS s : le résultat est perdu
s = s.toUpperCase();      // s désigne maintenant une nouvelle chaîne "BONJOUR"
```

Ne pas confondre
Un tableau a une taille fixe, mais ses cases sont **modifiables** (`t[0] = 5`). L'immuabilité de `String` n'a rien à voir avec les tableaux : c'est un **choix de conception** de la classe, dont le contenu interne est privé et qu'aucune méthode ne modifie.

Pourquoi ce choix ? Une chaîne immuable peut être partagée sans risque entre plusieurs parties du programme ou plusieurs threads, Java peut réutiliser les chaînes identiques écrites dans le code, et elle fait une clé fiable dans un dictionnaire (`HashMap`).

Construire une chaîne morceau par morceau : StringBuilder

Comme chaque concaténation crée une nouvelle chaîne, concaténer dans une boucle est inefficace. On utilise alors `StringBuilder`, qui, lui, est modifiable :

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 5; i++) {
    sb.append(i).append(" ");
}
String resultat = sb.toString();     // "1 2 3 4 5 "
```

À retenir
Une `String` ne se modifie jamais : on récupère une nouvelle chaîne. On compare deux chaînes avec `equals()`, jamais avec `==`. Pour construire une chaîne dans une boucle, on utilise `StringBuilder`.

Testez-vous : Chapitre 7

1. Combien de cases contient `new double[10]` ? Quels sont le premier et le dernier indice ? Que contiennent les cases ?
2. Qu'affiche ce code ? `int[] t = {4, 8, 15}; System.out.println(t[t.length - 1]);`
3. Écrivez une boucle qui calcule le maximum d'un tableau d'entiers `t`.
4. Après `String s = "java"; s.toUpperCase();`, que vaut `s` ?
5. Que renvoie `"Bonjour".substring(3)` ?

Chapitre 8

Programmation orientée objet

La programmation orientée objet (POO) organise un programme autour d'objets qui regroupent des données et les opérations sur ces données. C'est le cœur de Java.

Objectifs du chapitre

- Définir une classe, créer des objets, écrire des constructeurs.
- Protéger les données d'un objet (encapsulation).
- Réutiliser une classe par héritage et redéfinir ses méthodes.
- Comprendre le polymorphisme, la surcharge et l'abstraction.

8.1 Les quatre piliers de la POO

Pilier	Idée	En Java
Encapsulation	Cacher l'état interne d'un objet et n'exposer que des méthodes contrôlées.	`private` + accesseurs (8.5)
Héritage	Réutiliser une classe existante et la spécialiser ; un objet de la sous-classe peut remplacer un objet de la classe parente.	`extends` (8.6)
Polymorphisme	Un même appel produit un comportement adapté à l'objet réel.	`@Override` (8.7)
Abstraction	Ne montrer que l'essentiel : un contrat, pas les détails.	`abstract`, `interface` (8.9)

Une analogie : une voiture. L'**encapsulation**, c'est le capot fermé : on utilise le volant et les pédales sans toucher au moteur. L'**héritage** : une voiture électrique reprend tout ce que sait faire une voiture et y ajoute la gestion de sa batterie. Le **polymorphisme** : « démarrer » ne fait pas la même chose sur une voiture thermique et une électrique. L'**abstraction** : le conducteur n'a pas besoin de connaître la mécanique pour conduire.

8.2 Classe et objet

Une **classe** est un modèle, comme le plan d'un architecte. Un **objet** est une réalisation concrète de ce modèle, comme une maison construite d'après le plan. On dit qu'un objet est une **instance** de sa classe. Plusieurs objets partagent le même plan, mais chacun a son propre état.

Une classe contient des **attributs** (l'état : ce que l'objet sait) et des **méthodes** (le comportement : ce que l'objet sait faire).

```
class Chien {
    String nom;                     // attribut : l'état

    void aboyer() {                 // méthode : le comportement
        System.out.println(nom + " : Wouf !");
    }
}

// Dans main :
Chien rex = new Chien();            // crée un objet
rex.nom = "Rex";
rex.aboyer();                       // affiche « Rex : Wouf ! »

Chien max = new Chien();            // un second objet, avec son propre nom
max.nom = "Max";
max.aboyer();                       // affiche « Max : Wouf ! »
```

À retenir

`new` alloue l'objet en mémoire, appelle un constructeur et renvoie une **référence** vers l'objet créé. La variable `rex` ne contient pas l'objet : elle le **désigne** (voir 3.2). Si l'on écrit `Chien autre = rex;`, les deux variables désignent le même chien.

8.3 Les constructeurs

Un **constructeur** est un bloc de code spécial, appelé automatiquement par `new`, qui sert à initialiser l'objet. Il porte le nom de la classe et n'a **aucun type de retour**, pas même `void`.

```
class Marque {
    String nom;
    String dateCreation;

    Marque(String nom, String dateCreation) {    // constructeur à deux paramètres
        this.nom = nom;
        this.dateCreation = dateCreation;
    }

    Marque(String nom) {                         // constructeur à un paramètre
        this(nom, "non défini");                 // appelle l'autre constructeur
    }
}

Marque a = new Marque("Apple", "01/04/1976");
Marque h = new Marque("Huawei");                 // dateCreation vaut "non défini"
```

Une classe peut avoir plusieurs constructeurs, à condition que leurs listes de paramètres diffèrent (c'est une surcharge, voir 8.8). `this(…)` appelle un autre constructeur de la même classe ; il doit être la première instruction.

Critère	Constructeur	Méthode
Nom	exactement celui de la classe	libre (un verbe en camelCase)
Retour	aucun type, pas même `void`	obligatoire (`void` si rien)
Appel	automatique, une fois par objet, via `new`	autant de fois que voulu
Héritage	non hérité ; appelé via `super(…)`	héritée, redéfinissable

Piège : le constructeur par défaut
Le « constructeur par défaut » est un constructeur sans paramètre que le compilateur ajoute **seulement si la classe n'en déclare aucun**. Dès que vous écrivez un constructeur avec paramètres, il disparaît : `new Marque()` ne compile plus, sauf si vous écrivez vous-même un constructeur sans argument.

8.4 Application : Marque et Téléphone

Un objet peut contenir d'autres objets comme attributs. Ici, chaque téléphone contient une référence vers un objet `Marque` :

```
class Telephone {
    String nom;
    Marque marque;       // un objet Marque comme attribut
    int numero;

    Telephone(String nom, Marque marque, int numero) {
        this.nom = nom;
        this.marque = marque;
        this.numero = numero;
    }

    void afficherInfos() {
        System.out.println("Marque : " + marque.nom);
        System.out.println("Nom : " + nom);
        System.out.println("Numéro : " + numero);
    }
}

Marque apple = new Marque("Apple", "01/04/1976");
Telephone iphone = new Telephone("iPhone X", apple, 17541);
iphone.afficherInfos();
```

C'est une **composition** : la classe `Telephone` utilise un objet `Marque` comme composant et lui confie tout ce qui concerne la marque. Elle se distingue de l'héritage (8.6) : `Telephone` ne reprend aucune méthode de `Marque` et un téléphone ne pourrait pas être transmis à une méthode qui attend une marque.

Pièges vus dans cet exemple

- `static` ne rend pas un champ « visible partout » : il le rend commun à tous les objets de la classe. La visibilité dépend de `private`, `protected`, `public` (3.6).
- Si `Marque` déclare `private String naissanceAuteur;`, alors `Marque.naissanceAuteur` écrit dans une autre classe ne compile pas, pour deux raisons : le champ est `private` (inaccessible hors de `Marque`) et il n'est pas `static` (il faut passer par un objet). La solution : une méthode publique `getNaissanceAuteur()`.

8.5 Encapsulation : protéger l'état

Si les attributs d'un objet sont accessibles depuis n'importe où, n'importe quelle partie du programme peut les mettre dans un état absurde : `compte.solde = -500;`. L'encapsulation consiste à rendre les attributs `private` et à n'autoriser leur modification que par des méthodes qui vérifient les règles.

```
public class Compte {
    private double solde;                        // invisible de l'extérieur

    public double getSolde() {                   // accesseur en lecture (getter)
        return solde;
    }

    public void deposer(double montant) {
        if (montant <= 0) {
            throw new IllegalArgumentException("Le montant doit être positif");
        }
        solde += montant;
    }

    public void retirer(double montant) {
        if (montant <= 0 || montant > solde) {
            throw new IllegalArgumentException("Retrait impossible");
        }
        solde -= montant;
    }
}
```

- **Attributs `private`** : personne ne peut écrire `compte.solde = -500` depuis une autre classe.
- **Méthodes `public` contrôlées** : chaque modification passe par une règle (un dépôt est positif, on ne retire pas plus que le solde).
- **Getters et setters** : `getSolde()` permet de lire. On ne crée un setter (`setSolde`) que si une modification libre a un sens, ce qui n'est pas le cas ici.

Autre bénéfice : on peut changer l'implémentation interne (stocker le solde en centimes, ajouter un historique) sans modifier le code qui utilise la classe.

8.6 L'héritage : spécialiser une classe

L'héritage permet de créer une classe à partir d'une autre : la nouvelle classe récupère les attributs et méthodes de la première, et peut en ajouter ou en modifier.

- La classe dont on hérite est la **super-classe** (ou classe parente, classe de base).
- La classe qui hérite est la **sous-classe** (ou classe enfant, classe dérivée).

Héritage ou composition : le principe de substitution

On n'écrit `class B extends A` que si un objet `B` peut être utilisé **partout où un `A` est attendu**, sans que le code qui manipule des `A` s'en aperçoive : `B` accepte les mêmes appels, respecte les mêmes règles et se contente d'ajouter ou de préciser des comportements. Un vélo tout-terrain peut être passé à n'importe quelle méthode qui accélère ou freine un `Velo` : l'héritage convient. Si la nouvelle classe a seulement **besoin des services** d'une autre, on en fait un attribut : c'est une composition (8.4). En cas de doute, la composition est le choix le plus souple.

```
class Velo {
    int vitesse;
    String numero;

    Velo(int vitesse, String numero) {
        this.vitesse = vitesse;
        this.numero = numero;
    }

    void accelerer(int increment) { vitesse += increment; }
    void freiner(int decrement)   { vitesse -= decrement; }

    String getInfo() {
        return "Numéro : " + numero + "\nVitesse : " + vitesse;
    }
}

class VeloToutTerrain extends Velo {           // VeloToutTerrain hérite de Velo
    String couleur;
    int hauteurSelle;

    VeloToutTerrain(String numero, int vitesse, String couleur, int hauteurSelle) {
        super(vitesse, numero);                // appelle le constructeur de Velo
        this.couleur = couleur;
        this.hauteurSelle = hauteurSelle;
    }

    @Override                                  // on redéfinit getInfo()
    String getInfo() {
        return super.getInfo() + "\nCouleur : " + couleur
             + "\nHauteur de selle : " + hauteurSelle;
    }
}

// Dans main :
VeloToutTerrain vtt = new VeloToutTerrain("745SV", 40, "Bleu", 7);
vtt.accelerer(100);                // méthode héritée de Velo, non redéfinie
System.out.println(vtt.getInfo()); // vitesse : 140, puis couleur et selle
```

Les règles de l'héritage

- **Héritage simple** : une classe n'étend qu'une seule classe. Elle peut en revanche implémenter plusieurs interfaces (8.9).
- Toute classe hérite, directement ou non, de la classe **`Object`**, qui fournit notamment `toString()` et `equals()`.
- **Les constructeurs ne s'héritent pas.** Le constructeur de la sous-classe appelle celui du parent avec `super(…)`, en **première instruction**.
- Les membres `private` du parent existent dans l'objet, mais la sous-classe ne peut pas y accéder directement.
- Une classe `final` ne peut pas avoir de sous-classe (exemple : `String`).

Redéfinir une méthode et `@Override`

La sous-classe peut **redéfinir** une méthode héritée : elle écrit une méthode de même nom et mêmes paramètres, qui remplace celle du parent. Dans l'exemple, `getInfo()` est redéfinie ; `super.getInfo()` appelle la version du parent pour réutiliser son code au lieu de le recopier.

L'annotation `@Override` demande au compilateur de vérifier qu'une méthode de même signature existe bien chez le parent. Sans elle, une faute de frappe (`getinfo` au lieu de `getInfo`) créerait silencieusement une nouvelle méthode au lieu de redéfinir l'ancienne. On redéfinit une **méthode**, jamais une classe.

8.7 Le polymorphisme

> **Définition**
> Le polymorphisme est la capacité d'**un même appel de méthode** à produire **des comportements différents** selon les types en jeu.

Il en existe deux formes :

Polymorphisme statique : la surcharge

Plusieurs méthodes portent le même nom mais ont des paramètres différents (en nombre ou en type). C'est le **compilateur** qui choisit laquelle appeler, d'après les arguments :

```
static int    somme(int a, int b)       { return a + b; }
static double somme(double a, double b) { return a + b; }
static int    somme(int a, int b, int c) { return a + b + c; }

somme(2, 3);          // appelle la version (int, int)
somme(2.5, 1.0);      // appelle la version (double, double)
somme(1, 2, 3);       // appelle la version à trois paramètres
```

Polymorphisme dynamique : la redéfinition

C'est le plus important en POO. Une variable dont le type est la super-classe peut désigner un objet d'une sous-classe. Lors d'un appel, c'est la **JVM** qui choisit, à l'exécution, la méthode de **l'objet réel**, et non celle du type de la variable :

```
Velo v = new VeloToutTerrain("745SV", 40, "Bleu", 7);
System.out.println(v.getInfo());    // version de VeloToutTerrain !

Velo[] parc = {
    new Velo(20, "A1"),
    new VeloToutTerrain("B2", 30, "Rouge", 5)
};
for (Velo x : parc) {
    System.out.println(x.getInfo());   // chaque vélo répond à sa manière
}
```

Ce mécanisme s'appelle la **liaison dynamique** (*dynamic dispatch*). Il permet d'écrire un seul code (la boucle ci-dessus) qui fonctionne avec toutes les sortes de vélos, y compris celles qui seront créées plus tard.

8.8 Surcharge ou redéfinition ?

Critère	Surcharge (*overloading*)	Redéfinition (*overriding*)
Où	dans une même classe (ou entre parent et enfant)	dans une sous-classe
Signature	même nom, paramètres différents	même nom, mêmes paramètres
Type de retour	libre, mais ne suffit pas à distinguer deux méthodes	identique ou sous-type
Visibilité	libre	égale ou plus large, jamais réduite
Choix fait	à la compilation	à l'exécution
`static`, `private`	surchargeables	non redéfinissables
Annotation	aucune	`@Override` recommandée

> **Piège**
> `int f()` et `double f()` dans la même classe ne compilent pas : le type de retour seul ne suffit pas à surcharger une méthode. Ce sont les **paramètres** qui doivent différer.

8.9 Abstraction : classes abstraites et interfaces

Parfois, une classe représente une idée trop générale pour être instanciée. Une « forme » n'existe pas en soi : on ne rencontre que des cercles, des rectangles… Mais toutes les formes savent calculer leur aire.

```
abstract class Forme {
    abstract double aire();                 // méthode abstraite : pas de corps

    void afficher() {                       // méthode concrète
        System.out.println("Aire : " + aire());
    }
}

class Cercle extends Forme {
    private double r;
    Cercle(double r) { this.r = r; }

    @Override
    double aire() { return Math.PI * r * r; }
}

class Rectangle extends Forme {
    private double l, h;
    Rectangle(double l, double h) { this.l = l; this.h = h; }

    @Override
    double aire() { return l * h; }
}

// new Forme() est interdit ; mais :
Forme[] formes = { new Cercle(1), new Rectangle(2, 3) };
for (Forme f : formes) f.afficher();       // polymorphisme
```

Les interfaces

Une **interface** est un contrat : une liste de méthodes qu'une classe s'engage à fournir. Une classe peut implémenter **plusieurs** interfaces.

```
interface Imprimable {
    void imprimer();
}

class Facture implements Imprimable {
    @Override
    public void imprimer() { System.out.println("Facture n° 42"); }
}

class Cercle2 extends Forme implements Imprimable { … }   // hérite ET implémente
```

	Classe abstraite	**Interface**
Instanciable ?	non	non
Attributs	oui, comme une classe normale	seulement des constantes
Constructeurs	oui	non
Combien par classe ?	une seule (`extends`)	plusieurs (`implements`)
Usage typique	factoriser le code commun d'une famille de classes	définir une capacité (« sait s'imprimer »)

Les interfaces sont la réponse de Java à l'héritage multiple, interdit pour les classes. Depuis Java 8, une interface peut aussi contenir des méthodes `default` avec un corps.

Testez-vous : Chapitre 8

1. Quelle est la différence entre une classe et un objet ?
2. Une classe déclare seulement `Point(int x, int y)`. Peut-on écrire `new Point()` ? Pourquoi ?
3. On dispose des classes `Forme`, `Carre`, `Voiture` et `Moteur`. Quelles relations écririez-vous entre elles, héritage ou composition ? Justifiez avec le principe de substitution.
4. Pourquoi rend-on les attributs `private` ?
5. `Velo v = new VeloToutTerrain(…); v.getInfo();` : quelle version de `getInfo()` s'exécute ? Comment s'appelle ce mécanisme ?
6. Peut-on instancier une classe abstraite ? Une classe peut-elle implémenter deux interfaces ?

Chapitre 9

TD 2 : exercices

Dix problèmes de synthèse sur les tableaux, les chaînes de caractères et la programmation orientée objet : encapsulation, constructeurs, héritage, classes abstraites, interfaces et polymorphisme. Chaque problème est conçu pour être traité en temps limité, comme un sujet d'examen. Les corrigés commentés figurent en annexe A.

9.1 Exercice 1 : bulletins de notes

Une classe suit quatre matières avec des coefficients : Mathématiques (4), Java (5), Anglais (2), Réseaux (3).

1. Écrivez la classe `Etudiant` : un nom et un tableau de notes, une note par matière. La méthode `setNote(int matiere, double valeur)` refuse toute note hors de l'intervalle [0 ; 20]. La méthode `moyenne(double[] coefs)` renvoie la moyenne pondérée.
2. Écrivez la classe `Classe` : les noms des matières, leurs coefficients (le constructeur vérifie qu'il y a autant de coefficients que de matières) et un tableau d'étudiants de capacité fixe. La méthode `inscrire(String nom)` crée l'étudiant, l'ajoute et le renvoie.
3. Écrivez `static String mention(double moy)` : Très bien (16 et plus), Bien (14), Assez bien (12), Passable (10), Ajourné en dessous de 10.
4. Écrivez `classer()`, qui trie les étudiants par moyenne décroissante à l'aide d'un **tri par sélection** programmé à la main, puis `afficherClassement()`. Les ex aequo partagent le même rang et le rang suivant est sauté (1er, 2e, 2e, 4e…). Attention à la comparaison de deux `double`.
5. Pour chaque matière, affichez la moyenne de la classe et la meilleure note avec le nom de l'étudiant qui l'a obtenue. Affichez enfin le taux de réussite (moyenne au moins égale à 10).

Données de test (Maths, Java, Anglais, Réseaux) :

Étudiant	Notes	Étudiant	Notes
Adjovi	14, 16, 12, 11	Dossou	12, 11, 9, 13
Bello	9, 8.5, 15, 10	Eyo	14, 16, 12, 11
Chabi	17, 18, 14, 16	Fagla	6, 9, 11, 7

Contrôle : Chabi est premier avec 16,71 ; Adjovi et Eyo sont deuxièmes ex aequo avec 13,79 ; le suivant est classé 4e ; le taux de réussite est de 66,7 %.

9.2 Exercice 2 : gestion bancaire

Une agence gère deux sortes de comptes.

1. Écrivez la classe abstraite `Compte` : un numéro attribué automatiquement à partir de 1001 grâce à un compteur `static`, un titulaire, un solde et un historique des opérations (`ArrayList<String>`). Chaque opération, acceptée ou refusée, est inscrite dans l'historique avec son montant et le solde obtenu.
2. La méthode `deposer` refuse les montants négatifs ou nuls. La méthode `retirer` est commune à tous les comptes, mais elle s'appuie sur une méthode abstraite `retraitAutorise(double montant)` que chaque sous-classe définit.
3. La méthode `virer(Compte destinataire, double montant)` ne crédite le destinataire que si le débit a réussi ; un virement vers soi-même est refusé.
4. `CompteCourant` : un découvert autorisé ; le solde ne doit jamais descendre en dessous de l'opposé du découvert. En fin de mois, on prélève 500 F de frais de tenue de compte, puis, si le

solde est négatif, des agios au taux annuel de 15 % calculés sur un mois.

5. `CompteEpargne` : un taux d'intérêt annuel ; pas de découvert ; au plus **deux retraits par mois**, les virements sortants comptant comme des retraits. En fin de mois, les intérêts mensuels sont versés et le compteur de retraits est remis à zéro.
6. Écrivez la classe `Agence` : liste de comptes, recherche par numéro, traitement de fin de mois de tous les comptes et encours total. Où le polymorphisme intervient-il ?

Scénario de test : compte courant de 50 000 F avec un découvert de 100 000 F ; compte épargne de 200 000 F à 3,5 %. Retrait de 120 000 F sur le courant (accepté), puis de 40 000 F (refusé). Sur l'épargne : retrait de 10 000 F, virement de 30 000 F vers le courant, retrait de 5 000 F (refusé). Fin de mois, puis nouveau retrait de 5 000 F sur l'épargne (accepté). L'encours total final est de 114 460 F.

9.3 Exercice 3 : calcul matriciel

On écrit une classe `Matrice` **immuable** : une fois construite, une matrice n'est plus jamais modifiée ; chaque opération renvoie une nouvelle matrice.

1. Le constructeur reçoit un tableau `double[][]`. Il vérifie que toutes les lignes ont la même longueur et en fait une copie. Pourquoi la copie est-elle indispensable pour garantir l'immuabilité ? Pourquoi déclarer la classe `final` ?
2. Écrivez `static Matrice identite(int n)`, `plus`, `fois` et `transposee`. Les opérations impossibles (dimensions incompatibles) sont refusées.
3. Écrivez `boolean egale(Matrice b)` avec une tolérance de 10^{-9}, puis `toString()`.
4. Écrivez `Matrice puissance(int k)` par **exponentiation rapide** récursive : $A^0 = I$; si k est pair, $A^k = (A^{k/2})^2$; sinon $A^k = A \times A^{k-1}$. Combien de multiplications faut-il environ pour $k = 1\,000$, comparé à la méthode naïve ?
5. Écrivez `double determinant()` par développement récursif selon la première ligne, à l'aide d'une méthode privée qui construit la sous-matrice privée d'une ligne et d'une colonne.
6. Tests : avec $A = [[2, -1, 0], [1, 3, 2], [0, 1, 4]]$ et $B = [[1, 0, 2], [-1, 1, 0], [3, 2, 1]]$, vérifiez que $A \times I = A$, que $(AB)^T = B^T A^T$, et que $\det(AB) = \det(A) \times \det(B)$ ($24 \times (-9) = -216$).
7. Le terme de Fibonacci $F(n)$ se lit en haut à droite de $[[1, 1], [1, 0]]^n$. Calculez $F(40) = 102\,334\,155$.

9.4 Exercice 4 : agence de location de véhicules

1. Déclarez une interface `Louable` avec la méthode `double tarif(int jours)`.
2. Écrivez la classe abstraite `Vehicule` qui implémente `Louable` : immatriculation, kilométrage, kilométrage lors de la dernière révision, disponibilité. Elle déclare deux méthodes abstraites, `prixJournalier()` et `intervalleRevision()` (en km), et définit un tarif dégressif commun : −10 % au-delà de 7 jours, −25 % au-delà de 30 jours.
3. Les méthodes `sortir()` et `rentrer(long kmParcourus)` refusent les opérations incohérentes : sortir un véhicule déjà loué, rentrer un véhicule qui n'était pas sorti, kilométrage négatif. Un véhicule doit être révisé dès que l'écart depuis la dernière révision atteint son intervalle.
4. Trois sous-classes :

Classe	Prix par jour	Révision tous les	Particularité
Voiture	25 000 F, ou 35 000 F au-delà de 5 places	15 000 km	nombre de places
Camion	60 000 F + 8 000 F par tonne de charge	30 000 km	facturé 2 jours au minimum
Moto	8 000 F	6 000 km	

5. Écrivez `AgenceLocation` : un parc (tableau), `louer(String immat, int jours)` qui renvoie le montant ou −1 si le véhicule est introuvable, déjà loué ou à réviser, `rendre`, le chiffre d'affaires

cumulé, et un **tri par insertion** du parc par kilométrage décroissant.

6. Expliquez pourquoi la règle du minimum de 2 jours du camion s'écrit en redéfinissant `tarif` et en appelant `super.tarif(…)`, sans recopier le calcul de la remise.

Contrôle : une voiture 5 places louée 10 jours coûte 225 000 F ; un camion de 3,5 t loué 1 jour coûte 176 000 F ; une moto à 5 900 km louée puis rendue après 6 100 km ne peut plus être louée avant révision.

9.5 Exercice 5 : analyse de texte et cryptographie

Écrivez une classe `AnalyseurTexte` construite à partir d'un texte. La méthode `split` est interdite.

1. `String[] mots()` découpe le texte en mots (suites de lettres ou de chiffres) en le parcourant caractère par caractère. Le tableau renvoyé a exactement la taille du nombre de mots.
2. `String motLePlusFrequent()` sans distinguer majuscules et minuscules, à l'aide de deux tableaux parallèles (mots distincts et nombre d'occurrences).
3. `String inverserMots()` : « Le chat dort » devient « dort chat Le ».
4. `static boolean estPalindrome(String s)` en ignorant la casse, les espaces et la ponctuation, avec deux indices qui se rapprochent. « Esope reste ici et se repose » est un palindrome.
5. `static boolean sontAnagrammes(String a, String b)` à l'aide d'un tableau de 26 compteurs, en un seul passage sur chaque chaîne.
6. **Chiffre de César** : `cesar(String s, int decalage)` décale chaque lettre en conservant la casse et laisse les autres caractères intacts. Le décalage peut être négatif : attention au comportement de `%` en Java avec les nombres négatifs.
7. **Chiffre de Vigenère** : chaque lettre est décalée de la valeur de la lettre correspondante de la clé (A = 0, B = 1…), la clé étant répétée et n'avançant que sur les lettres. Écrivez le chiffrement et le déchiffrement. « BONJOUR » chiffré avec la clé « CLE » donne « DZRLZYT ».
8. **Cryptanalyse** : en français, la lettre la plus fréquente est le E. Écrivez `devinerDecalage(String chiffre)` qui retrouve le décalage d'un message chiffré par César, et testez-la sur une phrase de votre choix. Dans quel cas cette méthode échoue-t-elle ?

9.6 Exercice 6 : morpion contre l'ordinateur

1. Écrivez la classe `Plateau` (tableau 3 × 3 de `char`) avec `estLibre`, `jouer`, `annuler`, `gagnant()` (renvoie le symbole gagnant ou un espace), `estPlein` et `afficher`. Lignes notées A à C, colonnes 1 à 3.
2. Écrivez la classe abstraite `Joueur` (nom, symbole) avec la méthode abstraite `int[] choisirCoup(Plateau p)`.
3. `Humain` lit une case de la forme « B2 » et la redemande tant qu'elle est invalide ou occupée.
4. `Ordinateur` applique, dans cet ordre, les règles suivantes : 1) gagner s'il le peut ; 2) sinon bloquer une victoire adverse ; 3) sinon créer une **fourchette** (un coup qui crée deux menaces de victoire à la fois) ; 4) sinon, si l'adversaire peut créer une fourchette, jouer un bord pour le forcer à se défendre ; 5) sinon prendre le centre ; 6) sinon un coin, puis un bord. Pour tester un coup, l'ordinateur le joue sur le plateau puis l'annule.
5. Écrivez `static Joueur partie(Joueur j1, Joueur j2, boolean afficher)` qui renvoie le gagnant ou `null` en cas de match nul. Grâce au polymorphisme, la même méthode fait jouer humain contre ordinateur, humain contre humain ou ordinateur contre ordinateur.
6. Vérifiez que deux ordinateurs font toujours match nul. Que montre ce test sur la stratégie ?

9.7 Exercice 7 : gestion de commandes et facturation

1. `Produit` : référence, désignation, prix HT strictement positif, stock positif ou nul. La méthode `prixUnitaire()` renvoie le prix réellement appliqué et `sortirDuStock(int qte)` refuse une quantité supérieure au stock.
2. `ProduitPerissable` hérite de `Produit` et connaît le nombre de jours avant péremption. Il redéfinit `prixUnitaire()` : −50 % à 2 jours ou moins, −20 % à 5 jours ou moins. Un produit à 0 jour est périmé et ne peut plus être commandé.
3. `LigneCommande` : un produit et une quantité, avec `sousTotal()`.
4. `Commande` : numéro automatique, client, liste de lignes. La méthode `ajouter(Produit p, int qte)` refuse une quantité nulle ou négative, un produit périmé, et toute quantité qui, ajoutée à celle déjà commandée pour ce produit, dépasserait le stock.
5. Remise sur le total brut HT : 5 % à partir de 100 000 F, 10 % à partir de 500 000 F ; puis TVA de 18 % sur le total après remise.
6. `valider()` est **tout ou rien** : on vérifie d'abord toutes les lignes, et on ne déstocke que si toutes sont satisfaisables. Pourquoi ne faut-il pas déstocker ligne par ligne au fur et à mesure ? Après validation, la commande n'est plus modifiable.
7. `imprimerFacture()` affiche les lignes, le total brut, la remise, le total HT, la TVA et le net à payer.

Contrôle : clavier à 7 500 F (stock 40), écran à 145 000 F (stock 3), yaourt à 2 400 F à 2 jours de péremption (stock 100), lait périmé. On commande 20 claviers, 4 écrans (refusé), 3 écrans, 10 yaourts et 5 laits (refusé). Total brut 597 000 F, remise 10 %, net à payer 634 014 F. Après validation, le stock d'écrans est nul.

9.8 Exercice 8 : simulation de file d'attente

Une agence se demande combien de guichets ouvrir. On simule une journée de 480 minutes.

1. Écrivez une classe `FileCirculaire` d'entiers, de capacité fixe, implémentée avec un tableau, un indice de début et une taille : `enfiler`, `defiler`, `estVide`, `estPleine`, `taille`. Quand la fin atteint le bout du tableau, elle repart au début : utilisez l'opérateur `%`. Pourquoi cette solution est-elle préférable au décalage de tous les éléments à chaque retrait ?
2. Écrivez une classe `Simulation` paramétrée par le nombre de guichets, la probabilité d'arrivée d'un client par minute, la durée, la capacité de la file et une graine aléatoire. Chaque minute : un client arrive avec la probabilité donnée (il repart si la file est pleine) ; on enfile sa minute d'arrivée ; chaque guichet libre prend le client en tête de file et l'occupe pendant 2 à 6 minutes, tirées au hasard.
3. Relevez le nombre de clients servis et refusés, l'attente moyenne et maximale, et la longueur maximale de la file.
4. Comparez 1, 2 et 3 guichets avec une probabilité d'arrivée de 0,3, une file de 30 places et la graine 2022. Pourquoi utiliser la même graine pour les trois simulations ?
5. Interprétez : avec un seul guichet, l'attente moyenne dépasse 30 minutes ; elle tombe à moins d'une minute avec deux guichets. Expliquez cet écart à partir de la durée moyenne de service (4 minutes) et du nombre moyen d'arrivées par minute.

9.9 Exercice 9 : bataille navale

1. Écrivez la classe `Navire` : nom, taille, position (ligne, colonne, orientation) et nombre de cases touchées, avec `occupe(int l, int c)`, `toucher()` et `estCoule()`.
2. Écrivez la classe `Grille` de 10 × 10 cases, qui mémorise pour chaque case le navire qui l'occupe (ou `null`) et si elle a déjà été visée. La flotte est composée d'un porte-avions (5 cases), d'un

croiseur (4), d'un contre-torpilleur (3), d'un sous-marin (3) et d'un torpilleur (2).

3. Écrivez `placerAleatoirement(Random r)` : chaque navire est placé horizontalement ou verticalement, sans sortir de la grille ni chevaucher un autre navire.
4. Écrivez `static int[] lireCase(String s)` qui convertit « B7 » ou « J10 » en indices, et renvoie `null` pour une saisie invalide comme « K3 » ou « B11 ».
5. Écrivez `String tirer(int l, int c)`, qui renvoie « À l'eau », « Touché », « Touché-coulé : » suivi du nom du navire, ou « Déjà joué », puis `flotteCoulee()` et un affichage de la grille (`.` case inconnue, `o` à l'eau, `X` touché).
6. Écrivez un programme de jeu au clavier qui compte les tirs. Puis écrivez un tireur automatique qui vise d'abord une case sur deux, en damier. Pourquoi est-on sûr de toucher chaque navire lors de ce premier passage ?

9.10 Exercice 10 : paie et organigramme d'entreprise

1. Écrivez la classe abstraite `Employe` : matricule automatique, nom, méthode abstraite `salaireBrut()`. Le salaire net vaut le brut diminué de 3,6 % de cotisation sociale et de l'impôt mensuel.
2. L'impôt est **progressif** : chaque tranche n'est taxée qu'à son propre taux.

Tranche du salaire brut mensuel	Taux
jusqu'à 60 000 F	0 %
de 60 000 à 150 000 F	10 %
de 150 000 à 300 000 F	15 %
au-delà de 300 000 F	25 %

   Vérifiez que l'impôt sur 200 000 F vaut 16 500 F.
3. Sous-classes : `Salarie` (salaire de base majoré de 2 % par année d'ancienneté, majoration plafonnée à 30 %) ; `Commercial`, qui perçoit la rémunération d'un salarié augmentée de 4 % de son chiffre d'affaires ; `Stagiaire`, qui reçoit une gratification ni imposée ni soumise à cotisation.
4. `Manager` : rémunéré comme un salarié, il dirige une équipe d'employés, parmi lesquels peuvent se trouver d'autres managers. Son brut comprend en plus 5 000 F par subordonné direct et 1 % de la masse salariale brute de toute son équipe, sous-équipes comprises. Un manager ne peut pas s'ajouter à sa propre équipe.
5. Écrivez, de façon **récursive** et en vous appuyant sur la redéfinition : `masseEquipe()` (masse brute d'un employé et de toute son équipe), `afficherOrganigramme(int niveau)` avec une indentation par niveau, et `mieuxPaye()` qui renvoie l'employé de plus haut salaire net de la sous-arborescence. Pourquoi aucune de ces méthodes n'a-t-elle besoin de tester le type réel des employés ?

Données de test : la directrice générale (base 600 000 F, 12 ans) dirige deux directeurs. Le directeur commercial (400 000 F, 8 ans) dirige deux commerciaux : 150 000 F, 3 ans, 9 000 000 F de chiffre d'affaires ; 150 000 F, 1 an, 2 500 000 F. La directrice technique (450 000 F, 5 ans) dirige deux salariés (300 000 F, 20 ans ; 250 000 F, 2 ans) et un stagiaire (50 000 F). La masse salariale brute totale vaut 3 249 427 F.

Annexe A

Corrigés des TD

Corrigés commentés des dix exercices du TD 1 (chapitre 6) et des dix exercices du TD 2 (chapitre 9). Tous les programmes ont été compilés et exécutés ; les valeurs de contrôle indiquées dans les énoncés proviennent de ces exécutions. D'autres solutions sont évidemment possibles.

A.1 TD 1 · Exercice 1 : facture d'électricité

Énoncé : section 6.1.

Points clés

- Le calcul par tranches se fait avec des `Math.min` successifs : chaque tranche ne facture que la partie de la consommation qui lui appartient.
- `montantTaxable` s'obtient sans dupliquer le barème : c'est le montant de l'énergie moins ce que coûte la tranche sociale.
- Deux boucles `do … while` imbriquées dans la boucle principale assurent la validation des index ; la sentinelle −1 est testée avant toute autre vérification.
- Les statistiques finales sont protégées contre la division par zéro quand aucun client n'a été saisi.
- Contrôle : 15, 300 et 100 kWh donnent un total facturé de 56 115 F.

```
import java.util.Scanner;

public class Facture {
    static final double PRIX_T1 = 76, PRIX_T2 = 112, PRIX_T3 = 135;   // FCFA par
kWh
    static final int LIMITE_T1 = 20, LIMITE_T2 = 250;                 // bornes des
tranches (kWh)
    static final double PRIME_FIXE = 1000, TAUX_TVA = 0.18;

    // Montant de l'énergie, calculé tranche par tranche (tarification progressive)
    static double montantEnergie(int kwh) {
        double montant = 0;
        if (kwh > LIMITE_T2) {
            montant += (kwh - LIMITE_T2) * PRIX_T3;
            kwh = LIMITE_T2;
        }
        if (kwh > LIMITE_T1) {
            montant += (kwh - LIMITE_T1) * PRIX_T2;
            kwh = LIMITE_T1;
        }
        return montant + kwh * PRIX_T1;
    }

    // Seule la partie située au-delà de la tranche sociale est soumise à la TVA
    static double montantTaxable(int kwh) {
        return montantEnergie(kwh) - montantEnergie(Math.min(kwh, LIMITE_T1));
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int nbClients = 0, nbSociaux = 0, consoMax = -1;
        double total = 0;

        while (true) {
```

```
            System.out.print("Ancien index (-1 pour terminer) : ");
            int ancien = sc.nextInt();
            if (ancien == -1) break;
            if (ancien < 0) {
                System.out.println("Index invalide.");
                continue;
            }
            int nouveau;
            do {
                System.out.print("Nouvel index : ");
                nouveau = sc.nextInt();
                if (nouveau < ancien) System.out.println("Le nouvel index doit être
supérieur ou égal à l'ancien.");
            } while (nouveau < ancien);

            int conso = nouveau - ancien;
            double energie = montantEnergie(conso);
            double tva = montantTaxable(conso) * TAUX_TVA;
            double net = energie + tva + PRIME_FIXE;
            System.out.printf("Consommation : %d kWh%n", conso);
            System.out.printf("Énergie : %.0f F   TVA : %.0f F   Prime fixe : %.0f
F%n", energie, tva, PRIME_FIXE);
            System.out.printf("Net à payer : %.0f F%n%n", net);

            nbClients++;
            total += net;
            if (conso <= LIMITE_T1) nbSociaux++;
            if (conso > consoMax) consoMax = conso;
        }

        if (nbClients == 0) {
            System.out.println("Aucun client saisi.");
        } else {
            System.out.printf("Clients : %d   Total facturé : %.0f F   Facture
moyenne : %.0f F%n",
                    nbClients, total, total / nbClients);
            System.out.printf("Plus forte consommation : %d kWh   Clients en
tranche sociale : %.1f %%%n",
                    consoMax, 100.0 * nbSociaux / nbClients);
        }
        sc.close();
    }
}
```

A.2 TD 1 · Exercice 2 : nombres parfaits et nombres amicaux

Énoncé : section 6.2.

Points clés

- Les diviseurs vont par paires $(d, n/d)$: en testant jusqu'à $\sqrt{n}$, on récupère les deux à la fois. Il faut éviter de compter deux fois d lorsque $d^2 = n$.
- Le rapport $s(n)/n$ doit être calculé en `double`, sinon la division entière fausse la comparaison.
- Pour les amicaux, on calcule $b = s(a)$ puis on vérifie $s(b) = a$; la condition $a < b$ affiche chaque couple une fois et exclut les parfaits ($a = b$).
- Résultats jusqu'à 10 000 : parfaits 6, 28, 496, 8128 ; 2 488 abondants ; 7 508 déficients ; le plus abondant est 5 040 (rapport 2,838) ; 5 couples amicaux.

```
public class NombresParfaits {

    // Somme des diviseurs de n strictement inférieurs à n, en ne testant que
jusqu'à racine de n
    static long sommeDiviseursPropres(int n) {
        if (n == 1) return 0;
        long somme = 1;                          // 1 divise toujours n
        for (int d = 2; (long) d * d <= n; d++) {
            if (n % d == 0) {
                somme += d;
                int associe = n / d;             // les diviseurs vont par paires
(d, n/d)
                if (associe != d) somme += associe;
            }
        }
        return somme;
    }

    static String nature(int n) {
        long s = sommeDiviseursPropres(n);
        if (s == n) return "parfait";
        return (s > n) ? "abondant" : "déficient";
    }

    public static void main(String[] args) {
        final int N = 10_000;
        int parfaits = 0, abondants = 0, deficients = 0;
        int plusAbondant = 1;
        double meilleurRatio = 0;

        System.out.print("Nombres parfaits jusqu'à " + N + " :");
        for (int n = 1; n <= N; n++) {
            long s = sommeDiviseursPropres(n);
            if (s == n) {
                parfaits++;
                System.out.print(" " + n);
            } else if (s > n) {
                abondants++;
            } else {
                deficients++;
            }
            double ratio = (double) s / n;
            if (ratio > meilleurRatio) {
                meilleurRatio = ratio;
                plusAbondant = n;
            }
        }
        System.out.println();
        System.out.println("Parfaits : " + parfaits + "   Abondants : " + abondants
+ "   Déficients : " + deficients);
        System.out.printf("Nombre le plus abondant : %d (somme / n = %.3f)%n",
plusAbondant, meilleurRatio);

        System.out.println("Couples de nombres amicaux :");
        for (int a = 2; a <= N; a++) {
            long b = sommeDiviseursPropres(a);
            // a < b évite d'afficher deux fois le même couple et exclut les
nombres parfaits
            if (b > a && b <= N && sommeDiviseursPropres((int) b) == a) {
                System.out.println("  (" + a + ", " + b + ")");
```

```
            }
        }
        System.out.println("Nature de 945 : " + nature(945) + ", de 12 : " +
nature(12) + ", de 28 : " + nature(28));
    }
}
```

A.3 TD 1 · Exercice 3 : changements de base

Énoncé : section 6.3.

Points clés

- `chiffre` repose sur l'arithmétique des caractères : `(char) ('0' + v)` ou `(char) ('A' + v - 10)`.
- Les restes des divisions successives donnent les chiffres de droite à gauche : on les ajoute donc **devant** la chaîne construite.
- Dans `binaireVersDecimal`, `% 10` extrait le dernier chiffre décimal et `/ 10` le retire ; un chiffre différent de 0 et de 1 provoque le retour de −1.
- Réflexion : un `int` ne contient que 10 chiffres décimaux, un `long` en contient 19. On peut donc saisir au plus 19 chiffres binaires, soit 111…1 (19 fois) = 524 287.
- Une puissance de 2 strictement positive a exactement un bit à 1.

```
import java.util.Scanner;

public class Bases {

    static char chiffre(int valeur) {
        return (valeur < 10) ? (char) ('0' + valeur) : (char) ('A' + valeur - 10);
    }

    // Divisions successives par la base : les restes donnent les chiffres, du
dernier au premier
    static String versBase(int n, int base) {
        if (n == 0) return "0";
        String resultat = "";
        while (n > 0) {
            resultat = chiffre(n % base) + resultat;
            n /= base;
        }
        return resultat;
    }

    // Lit un nombre écrit avec des 0 et des 1 (stocké dans un long) ; renvoie -1
si un chiffre est invalide
    static long binaireVersDecimal(long binaire) {
        long decimal = 0, poids = 1;
        while (binaire > 0) {
            long c = binaire % 10;
            if (c > 1) return -1;
            decimal += c * poids;
            poids *= 2;
            binaire /= 10;
        }
        return decimal;
    }

    static int bitsA1(int n) {
```

```
        int compte = 0;
        while (n > 0) {
            compte += n % 2;
            n /= 2;
        }
        return compte;
    }

    static boolean estPuissanceDe2(int n) {
        return n > 0 && bitsA1(n) == 1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choix;
        do {
            System.out.println("\n1 : décimal vers bases 2, 8, 16   2 : binaire
vers décimal   3 : analyse des bits   0 : quitter");
            choix = sc.nextInt();
            switch (choix) {
                case 1 -> {
                    System.out.print("Entier positif : ");
                    int n = sc.nextInt();
                    if (n < 0) System.out.println("Entier négatif refusé.");
                    else System.out.println("Base 2 : " + versBase(n, 2) + "   base
8 : " + versBase(n, 8)
                            + "   base 16 : " + versBase(n, 16));
                }
                case 2 -> {
                    System.out.print("Nombre binaire : ");
                    long b = sc.nextLong();
                    long d = binaireVersDecimal(b);
                    System.out.println(d < 0 ? "Ce n'est pas un nombre binaire." :
"Valeur décimale : " + d);
                }
                case 3 -> {
                    System.out.print("Entier positif : ");
                    int n = sc.nextInt();
                    System.out.println("Bits à 1 : " + bitsA1(n) + "   puissance de
2 : " + (estPuissanceDe2(n) ? "oui" : "non"));
                }
                case 0 -> System.out.println("Au revoir.");
                default -> System.out.println("Choix invalide.");
            }
        } while (choix != 0);
        sc.close();
    }
}
```

A.4 TD 1 · Exercice 4 : calendrier perpétuel

Énoncé : section 6.4.

Points clés

- `jourSemaine` transforme le résultat de Zeller (0 = samedi) en numérotation 1 = lundi par `(h + 5) % 7 + 1`.
- Pour l'écart entre deux dates sans tableau, on convertit chaque date en nombre de jours écoulés depuis une origine fixe, puis on soustrait.
- `afficherMois` commence par décaler la première ligne selon le jour du 1er du mois et passe à la ligne après chaque dimanche.
- Contrôles : le 10/01/2022 est un lundi, 10e jour de l'année ; il s'est écoulé 8 045 jours depuis le 01/01/2000.

```
import java.util.Scanner;

public class Calendrier {

    static boolean estBissextile(int a) {
        return (a % 4 == 0 && a % 100 != 0) || a % 400 == 0;
    }

    static int joursDansMois(int m, int a) {
        return switch (m) {
            case 4, 6, 9, 11 -> 30;
            case 2 -> estBissextile(a) ? 29 : 28;
            default -> 31;
        };
    }

    static boolean dateValide(int j, int m, int a) {
        return a >= 1583 && m >= 1 && m <= 12 && j >= 1 && j <= joursDansMois(m,
a);
    }

    // Congruence de Zeller (calendrier grégorien) ; renvoie 1 pour lundi … 7 pour
dimanche
    static int jourSemaine(int j, int m, int a) {
        if (m < 3) {          // janvier et février comptent comme les mois 13 et
14 de l'année précédente
            m += 12;
            a -= 1;
        }
        int k = a % 100, s = a / 100;
        int h = (j + 13 * (m + 1) / 5 + k + k / 4 + s / 4 + 5 * s) % 7;   // 0 =
samedi
        return (h + 5) % 7 + 1;
    }

    static String nomJour(int n) {
        return switch (n) {
            case 1 -> "lundi"; case 2 -> "mardi"; case 3 -> "mercredi"; case 4 ->
"jeudi";
            case 5 -> "vendredi"; case 6 -> "samedi"; default -> "dimanche";
        };
    }
```

```
    static int numeroDansAnnee(int j, int m, int a) {
        int n = j;
        for (int mois = 1; mois < m; mois++) n += joursDansMois(mois, a);
        return n;
    }

    // Nombre de jours écoulés depuis le 1er janvier 1583 (sert à calculer des
écarts)
    static long joursDepuisOrigine(int j, int m, int a) {
        long n = 0;
        for (int an = 1583; an < a; an++) n += estBissextile(an) ? 366 : 365;
        return n + numeroDansAnnee(j, m, a);
    }

    static void afficherMois(int m, int a) {
        System.out.println(" Lu Ma Me Je Ve Sa Di");
        int premier = jourSemaine(1, m, a);
        for (int i = 1; i < premier; i++) System.out.print("   ");
        for (int j = 1; j <= joursDansMois(m, a); j++) {
            System.out.printf("%3d", j);
            if ((premier + j - 1) % 7 == 0) System.out.println();
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int j1, m1, a1, j2, m2, a2;
        do {
            System.out.print("Première date (jour mois année) : ");
            j1 = sc.nextInt(); m1 = sc.nextInt(); a1 = sc.nextInt();
            if (!dateValide(j1, m1, a1)) System.out.println("Date invalide,
recommencez.");
        } while (!dateValide(j1, m1, a1));

        System.out.println("C'était un " + nomJour(jourSemaine(j1, m1, a1))
                + ", jour n° " + numeroDansAnnee(j1, m1, a1) + " de l'année.");
        afficherMois(m1, a1);

        do {
            System.out.print("Seconde date (jour mois année) : ");
            j2 = sc.nextInt(); m2 = sc.nextInt(); a2 = sc.nextInt();
            if (!dateValide(j2, m2, a2)) System.out.println("Date invalide,
recommencez.");
        } while (!dateValide(j2, m2, a2));

        long ecart = Math.abs(joursDepuisOrigine(j2, m2, a2) -
joursDepuisOrigine(j1, m1, a1));
        System.out.println("Écart entre les deux dates : " + ecart + " jours.");
        sc.close();
    }
}
```

A.5 TD 1 · Exercice 5 : tableau d'amortissement d'un prêt

Énoncé : section 6.5.

Points clés

- Le cas du taux nul doit être traité à part : la formule donnerait 0/0.
- Lors de la dernière échéance, le capital amorti est forcé à la valeur du capital restant : on absorbe ainsi les erreurs d'arrondi accumulées.
- `dureeMinimale` essaie les durées croissantes : la mensualité diminuant avec la durée, la première qui tient dans le budget est la plus courte.
- Contrôles : 1 000 000 F à 12 % sur 12 mois donnent 88 849 F par mois et 66 185 F d'intérêts ; avec un budget de 50 000 F, il faut au moins 23 mois.

```
import java.util.Scanner;

public class Pret {

    // Mensualité constante : M = C × t / (1 − (1 + t)^(−n)), t étant le taux
mensuel
    static double mensualite(double capital, double tauxAnnuel, int nbMois) {
        double t = tauxAnnuel / 100 / 12;
        if (t == 0) return capital / nbMois;
        return capital * t / (1 - Math.pow(1 + t, -nbMois));
    }

    static void tableauAmortissement(double capital, double tauxAnnuel, int nbMois)
{
        double t = tauxAnnuel / 100 / 12;
        double m = mensualite(capital, tauxAnnuel, nbMois);
        double restant = capital, totalInterets = 0;
        System.out.println(" Mois   Mensualité    Intérêts     Capital
Restant");
        for (int mois = 1; mois <= nbMois; mois++) {
            double interets = restant * t;
            double amorti = m - interets;
            if (mois == nbMois) amorti = restant;           // la dernière échéance
solde exactement le prêt
            restant -= amorti;
            totalInterets += interets;
            System.out.printf("%5d %12.0f %11.0f %11.0f %12.0f%n", mois, interets +
amorti, interets, amorti, Math.abs(restant));
        }
        System.out.printf("Total des intérêts : %.0f F   Coût total du crédit :
%.0f F%n", totalInterets, capital + totalInterets);
    }

    // Plus petite durée (en mois, au plus 360) dont la mensualité tient dans le
budget ; -1 si aucune
    static int dureeMinimale(double capital, double tauxAnnuel, double budget) {
        for (int n = 1; n <= 360; n++) {
            if (mensualite(capital, tauxAnnuel, n) <= budget) return n;
        }
        return -1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double capital, taux;
```

```
        int duree;
        do {
            System.out.print("Capital emprunté (F) : ");
            capital = sc.nextDouble();
        } while (capital <= 0);
        do {
            System.out.print("Taux annuel (%) : ");
            taux = sc.nextDouble();
        } while (taux < 0 || taux > 30);
        do {
            System.out.print("Durée (mois, 1 à 360) : ");
            duree = sc.nextInt();
        } while (duree < 1 || duree > 360);

        System.out.printf("Mensualité : %.0f F%n", mensualite(capital, taux,
duree));
        tableauAmortissement(capital, taux, duree);

        System.out.print("Budget mensuel maximal (F) : ");
        double budget = sc.nextDouble();
        int n = dureeMinimale(capital, taux, budget);
        if (n < 0) System.out.println("Impossible de rembourser ce prêt en 30 ans
avec ce budget.");
        else System.out.printf("Durée minimale : %d mois, mensualité %.0f F%n", n,
mensualite(capital, taux, n));
        sc.close();
    }
}
```

A.6 TD 1 · Exercice 6 : facteurs premiers, PGCD et Goldbach

Énoncé : section 6.6.

Points clés

- La décomposition divise *n* par chaque diviseur tant que c'est possible, en comptant l'exposant. Ce qui reste à la fin, s'il est supérieur à 1, est un dernier facteur premier.
- Euclide récursif : `pgcd(a, b) = (b == 0) ? a : pgcd(b, a % b)`.
- `a / pgcd(a, b) * b` divise avant de multiplier : le résultat intermédiaire reste petit et le risque de dépassement de capacité est réduit.
- Jusqu'à 1 000, le pair qui admet le plus de décompositions de Goldbach est 990, avec 52 décompositions.

```
import java.util.Scanner;

public class Facteurs {

    static boolean estPremier(int n) {
        if (n < 2) return false;
        for (int d = 2; (long) d * d <= n; d++) {
            if (n % d == 0) return false;
        }
        return true;
    }

    // Exemple : 360 -> "360 = 2^3 x 3^2 x 5"
    static String decomposition(int n) {
        String res = n + " =";
```

```
        int reste = n;
        boolean premierFacteur = true;
        for (int p = 2; (long) p * p <= reste; p++) {
            int exposant = 0;
            while (reste % p == 0) {
                reste /= p;
                exposant++;
            }
            if (exposant > 0) {
                res += (premierFacteur ? " " : " x ") + p + (exposant > 1 ? "^" +
exposant : "");
                premierFacteur = false;
            }
        }
        if (reste > 1) res += (premierFacteur ? " " : " x ") + reste;   // dernier
facteur, premier et plus grand que la racine
        return res;
    }

    // Algorithme d'Euclide, version récursive
    static long pgcd(long a, long b) {
        return (b == 0) ? a : pgcd(b, a % b);
    }

    static long ppcm(long a, long b) {
        return a / pgcd(a, b) * b;    // diviser avant de multiplier limite le
risque de dépassement
    }

    // Nombre de façons d'écrire n (pair) comme somme p + q de deux premiers, avec
p <= q
    static int nbDecompositionsGoldbach(int n) {
        int nb = 0;
        for (int p = 2; p <= n / 2; p++) {
            if (estPremier(p) && estPremier(n - p)) nb++;
        }
        return nb;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a, b;
        do {
            System.out.print("Deux entiers supérieurs à 1 : ");
            a = sc.nextInt();
            b = sc.nextInt();
        } while (a < 2 || b < 2);
        System.out.println(decomposition(a));
        System.out.println(decomposition(b));
        System.out.println("PGCD = " + pgcd(a, b) + "   PPCM = " + ppcm(a, b));
        System.out.println(pgcd(a, b) == 1 ? "Ils sont premiers entre eux." : "Ils
ne sont pas premiers entre eux.");

        System.out.print("Vérifier la conjecture de Goldbach jusqu'à : ");
        int limite = sc.nextInt();
        int champion = 4, maxDecomp = 0;
        boolean conjectureVerifiee = true;
        for (int n = 4; n <= limite; n += 2) {
            int d = nbDecompositionsGoldbach(n);
            if (d == 0) {
                conjectureVerifiee = false;
```

```
                System.out.println("Contre-exemple : " + n);
            }
            if (d > maxDecomp) {
                maxDecomp = d;
                champion = n;
            }
        }
        System.out.println(conjectureVerifiee ? "Conjecture vérifiée jusqu'à " +
limite + "." : "Conjecture mise en défaut !");
        System.out.println("Pair ayant le plus de décompositions : " + champion + "
(" + maxDecomp + ")");
        sc.close();
    }
}
```

A.7 TD 1 · Exercice 7 : le jeu des allumettes

Énoncé : section 6.7.

Points clés

- Positions perdantes : 1 (on doit prendre la dernière), puis 5, 9, 13… car depuis $4k + 1$, tout coup de 1 à 3 laisse à l'adversaire de quoi revenir à $4(k - 1) + 1$.
- Stratégie : jouer `(n - 1) % 4` allumettes. Si ce nombre vaut 0, on est en position perdante et l'ordinateur joue au hasard en espérant une erreur.
- La saisie est bornée par `Math.min(3, n)`.
- Avec 21 allumettes, $21 = 4 \times 5 + 1$ est une position perdante pour celui qui joue : il faut laisser l'adversaire commencer.

```
import java.util.Random;
import java.util.Scanner;

public class Nim {
    static final Random HASARD = new Random();

    static void afficher(int n) {
        String s = "";
        for (int i = 0; i < n; i++) s += "|";
        System.out.println(s + "  (" + n + ")");
    }

    // Les positions perdantes pour celui qui doit jouer sont 1, 5, 9, … (n % 4 ==
1).
    // L'ordinateur cherche donc à laisser un nombre d'allumettes de la forme 4k +
1.
    static int coupOrdinateur(int n) {
        int coup = (n - 1) % 4;
        if (coup == 0) coup = 1 + HASARD.nextInt(Math.min(3, n));   // position
perdante : on joue au hasard
        return coup;
    }

    static int coupJoueur(Scanner sc, int n) {
        int max = Math.min(3, n), coup;
        do {
            System.out.print("Combien d'allumettes prenez-vous (1 à " + max + ") ?
");
            coup = sc.nextInt();
```

```
        } while (coup < 1 || coup > max);
        return coup;
    }

    // Renvoie true si le joueur humain gagne
    static boolean partie(Scanner sc, int n, boolean joueurCommence) {
        boolean tourJoueur = joueurCommence;
        while (n > 0) {
            afficher(n);
            int coup = tourJoueur ? coupJoueur(sc, n) : coupOrdinateur(n);
            if (!tourJoueur) System.out.println("L'ordinateur prend " + coup +
"."); 
            n -= coup;
            if (n == 0) return !tourJoueur;   // celui qui vient de prendre la
dernière allumette a perdu
            tourJoueur = !tourJoueur;
        }
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int victoires = 0, defaites = 0;
        int rejouer;
        do {
            int n;
            do {
                System.out.print("Nombre d'allumettes au départ (10 à 50) : ");
                n = sc.nextInt();
            } while (n < 10 || n > 50);
            System.out.print("Voulez-vous commencer (1 = oui, 0 = non) ? ");
            boolean commence = sc.nextInt() == 1;
            if (partie(sc, n, commence)) {
                victoires++;
                System.out.println("Bravo, vous avez gagné !");
            } else {
                defaites++;
                System.out.println("Vous avez pris la dernière allumette :
perdu.");
            }
            System.out.println("Score : vous " + victoires + " - ordinateur " +
defaites);
            System.out.print("Rejouer (1 = oui, 0 = non) ? ");
            rejouer = sc.nextInt();
        } while (rejouer == 1);
        sc.close();
    }
}
```

A.8 TD 1 · Exercice 8 : relevé de températures

Énoncé : section 6.8.

Points clés

- Sans tableau, on cumule la somme et la somme des carrés ; la variance vaut alors moyenne des carrés moins carré de la moyenne (formule de König-Huygens). Le `Math.max(variance, 0)` protège des valeurs très légèrement négatives dues aux arrondis.
- Les pics demandent de garder les deux valeurs précédentes : le jour $j-1$ est un pic si sa valeur dépasse celle de $j-2$ et celle de j.
- La série de hausses courante est remise à 1 dès qu'une valeur n'est pas strictement supérieure à la précédente.
- Réflexion : compter les jours au-dessus de la moyenne est impossible en un seul passage sans mémoriser les valeurs, car la moyenne n'est connue qu'à la fin. Il faut soit un tableau, soit une seconde saisie.

```
import java.util.Scanner;

public class Releve {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n;
        do {
            System.out.print("Nombre de jours (au moins 3) : ");
            n = sc.nextInt();
        } while (n < 3);

        double somme = 0, sommeCarres = 0;
        double min = 0, max = 0;
        int jourMin = 1, jourMax = 1;
        int serie = 1, meilleureSerie = 1, debutSerie = 1, debutMeilleure = 1;
        int pics = 0;
        double avantDernier = 0, dernier = 0;     // les deux valeurs précédentes,
pour détecter les pics

        for (int jour = 1; jour <= n; jour++) {
            System.out.print("Température du jour " + jour + " : ");
            double t = sc.nextDouble();
            somme += t;
            sommeCarres += t * t;

            if (jour == 1 || t < min) { min = t; jourMin = jour; }
            if (jour == 1 || t > max) { max = t; jourMax = jour; }

            if (jour > 1) {
                if (t > dernier) {
                    serie++;
                } else {
                    serie = 1;
                    debutSerie = jour;
                }
                if (serie > meilleureSerie) {
                    meilleureSerie = serie;
                    debutMeilleure = debutSerie;
                }
            }
            // le jour précédent est un pic s'il dépasse ses deux voisins
```

```
            if (jour >= 3 && dernier > avantDernier && dernier > t) pics++;

            avantDernier = dernier;
            dernier = t;
        }

        double moyenne = somme / n;
        double variance = sommeCarres / n - moyenne * moyenne;   // formule de
König-Huygens
        double ecartType = Math.sqrt(Math.max(variance, 0));      // max(…, 0)
protège des arrondis négatifs

        System.out.printf("Moyenne : %.2f   Écart-type : %.2f%n", moyenne,
ecartType);
        System.out.printf("Minimum : %.1f (jour %d)   Maximum : %.1f (jour %d)%n",
min, jourMin, max, jourMax);
        System.out.println("Plus longue hausse : " + meilleureSerie + " jours, du
jour " + debutMeilleure
                + " au jour " + (debutMeilleure + meilleureSerie - 1));
        System.out.println("Nombre de pics : " + pics);
        sc.close();
    }
}
```

A.9 TD 1 · Exercice 9 : approximations numériques

Énoncé : section 6.9.

Points clés

- Leibniz et Wallis convergent toutes deux lentement : le nombre de termes est proportionnel à 1/précision (10 fois plus de termes pour une décimale de plus ; environ 100 000 pour 10^{-5}). Héron, au contraire, double à peu près le nombre de décimales exactes à chaque itération : 6 itérations suffisent pour √2. Le nombre d'itérations augmente avec a car on part de $x_0 = a$, loin de la racine.
- Dans la série de l'exponentielle, chaque terme vaut le précédent multiplié par x/k : aucune factorielle n'est calculée, donc pas de dépassement.
- Pour $x = -10$, les termes alternent de signe et atteignent environ 2 755 avant de décroître, alors que le résultat vaut $4{,}5 \times 10^{-5}$: les soustractions de grands nombres presque égaux détruisent les chiffres significatifs (phénomène de compensation). Calculer $1/e^{10}$ serait bien plus précis.

```
public class Approximations {

    // Leibniz : pi/4 = 1 − 1/3 + 1/5 − 1/7 + …  ; renvoie le nombre de termes pour
atteindre la précision
    static long termesLeibniz(double precision) {
        double somme = 0;
        long k = 0;
        while (Math.abs(4 * somme - Math.PI) >= precision) {
            somme += ((k % 2 == 0) ? 1.0 : -1.0) / (2 * k + 1);
            k++;
        }
        return k;
    }

    // Wallis : pi/2 = (2/1 × 2/3) × (4/3 × 4/5) × (6/5 × 6/7) × …
```

```
    static long facteursWallis(double precision) {
        double produit = 1;
        long k = 0;
        while (Math.abs(2 * produit - Math.PI) >= precision) {
            k++;
            double a = 2.0 * k;
            produit *= (a / (a - 1)) * (a / (a + 1));
        }
        return k;
    }

    // Méthode de Héron : x(k+1) = (x(k) + a / x(k)) / 2
    static double racineHeron(double a, double precision) {
        if (a == 0) return 0;
        double x = a, precedent;
        int iterations = 0;
        do {
            precedent = x;
            x = (x + a / x) / 2;
            iterations++;
        } while (Math.abs(x - precedent) >= precision);
        System.out.printf("  Héron(%.0f) : %d itérations%n", a, iterations);
        return x;
    }

    // Série de Taylor : e^x = somme de x^k / k!, chaque terme se déduit du
précédent (pas de factorielle)
    static double exponentielle(double x, double precision) {
        double somme = 1, terme = 1;
        int k = 0;
        while (Math.abs(terme) >= precision) {
            k++;
            terme *= x / k;
            somme += terme;
        }
        return somme;
    }

    public static void main(String[] args) {
        System.out.println("Précision    Leibniz (termes)   Wallis (facteurs)");
        for (double p = 1e-1; p >= 1e-5; p /= 10) {
            System.out.printf("%-12.0e %17d %19d%n", p, termesLeibniz(p),
facteursWallis(p));
        }
        for (double a = 2; a <= 2e6; a *= 1000) {       // 2, 2 000, 2 000 000
            double r = racineHeron(a, 1e-12);
            System.out.printf("  racine(%.0f) = %.12f   Math.sqrt : %.12f%n", a, r,
Math.sqrt(a));
        }
        for (double x = -10; x <= 20; x += 15) {         // -10, 5, 20
            System.out.printf("e^%.0f : série = %.10g   Math.exp = %.10g%n", x,
exponentielle(x, 1e-15), Math.exp(x));
        }
    }
}
```

A.10 TD 1 · Exercice 10 : la suite de Syracuse

Énoncé : section 6.10.

Points clés

- Toutes les valeurs de la suite sont des `long` : pour certains départs modestes, l'altitude dépasse la capacité d'un `int`.
- Le premier départ dont l'altitude dépasse `Integer.MAX_VALUE` est 113 383. En `int`, le calcul de $3n + 1$ déborderait sans erreur et donnerait un nombre négatif, puis une suite fausse, voire infinie.
- La recherche des records ne garde que la meilleure durée rencontrée jusque-là.
- Contrôle : pour 27, la durée du vol est de 111 étapes et l'altitude maximale de 9 232.

```
import java.util.Scanner;

public class Syracuse {

    static long suivant(long n) {
        return (n % 2 == 0) ? n / 2 : 3 * n + 1;
    }

    static int dureeVol(long n) {
        int etapes = 0;
        while (n != 1) {
            n = suivant(n);
            etapes++;
        }
        return etapes;
    }

    static long altitudeMax(long n) {
        long max = n;
        while (n != 1) {
            n = suivant(n);
            if (n > max) max = n;
        }
        return max;
    }

    // Nombre d'étapes pendant lesquelles la suite reste strictement au-dessus de
sa valeur de départ
    static int dureeEnAltitude(long n) {
        long depart = n;
        int etapes = 0;
        n = suivant(n);
        while (n > depart) {
            etapes++;
            n = suivant(n);
        }
        return etapes;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Nombre de départ : ");
        long n = sc.nextLong();
        String suite = "" + n;
        long x = n;
```

```
        while (x != 1) {
            x = suivant(x);
            suite += ", " + x;
        }
        System.out.println(suite);
        System.out.println("Durée du vol : " + dureeVol(n) + "   Altitude maximale
: " + altitudeMax(n)
                + "   Durée en altitude : " + dureeEnAltitude(n));

        System.out.print("Rechercher les records jusqu'à : ");
        int limite = sc.nextInt();
        int record = -1;
        long premierDepassement = -1;
        System.out.println("Records successifs de durée de vol :");
        for (int depart = 1; depart <= limite; depart++) {
            int d = dureeVol(depart);
            if (d > record) {
                record = d;
                System.out.println("  " + depart + " : " + d + " étapes");
            }
            if (premierDepassement < 0 && altitudeMax(depart) > Integer.MAX_VALUE)
premierDepassement = depart;
        }
        if (premierDepassement > 0) {
            System.out.println("Premier départ dont l'altitude dépasse la capacité
d'un int : " + premierDepassement);
        }
        sc.close();
    }
}
```

A.11 TD 2 · Exercice 1 : bulletins de notes

Énoncé : section 9.1.

Points clés

- La validation des notes est faite dans `setNote` : un étudiant ne peut jamais contenir une note invalide.
- Le tri par sélection cherche, à chaque tour, le meilleur des étudiants restants et l'échange avec la case courante.
- Les ex aequo sont détectés en comparant les moyennes avec une tolérance (10^{-9}) et non avec == ; le rang ne change que si la moyenne diffère, et il prend alors la valeur `i + 1`.
- Contrôle : Chabi 16,71 ; Adjovi et Eyo 13,79 (2e ex aequo) ; Dossou 4e ; taux de réussite 66,7 %.

```
class Etudiant {
    private final String nom;
    private final double[] notes;

    Etudiant(String nom, int nbMatieres) {
        this.nom = nom;
        this.notes = new double[nbMatieres];
    }

    String getNom() { return nom; }
    double getNote(int m) { return notes[m]; }
```

```
    void setNote(int m, double valeur) {
        if (valeur < 0 || valeur > 20) throw new IllegalArgumentException("Note
hors de [0 ; 20] : " + valeur);
        notes[m] = valeur;
    }

    double moyenne(double[] coefs) {
        double somme = 0, totalCoefs = 0;
        for (int m = 0; m < notes.length; m++) {
            somme += notes[m] * coefs[m];
            totalCoefs += coefs[m];
        }
        return somme / totalCoefs;
    }
}

class Classe {
    private final String[] matieres;
    private final double[] coefs;
    private final Etudiant[] etudiants;
    private int nb = 0;

    Classe(String[] matieres, double[] coefs, int capacite) {
        if (matieres.length != coefs.length) throw new
IllegalArgumentException("Une matière = un coefficient");
        this.matieres = matieres;
        this.coefs = coefs;
        this.etudiants = new Etudiant[capacite];
    }

    Etudiant inscrire(String nom) {
        if (nb == etudiants.length) throw new IllegalStateException("Classe
complète");
        Etudiant e = new Etudiant(nom, matieres.length);
        etudiants[nb++] = e;
        return e;
    }

    static String mention(double moy) {
        if (moy >= 16) return "Très bien";
        if (moy >= 14) return "Bien";
        if (moy >= 12) return "Assez bien";
        if (moy >= 10) return "Passable";
        return "Ajourné";
    }

    // Tri par sélection, du meilleur au moins bon (le tableau d'origine est
modifié)
    void classer() {
        for (int i = 0; i < nb - 1; i++) {
            int meilleur = i;
            for (int j = i + 1; j < nb; j++) {
                if (etudiants[j].moyenne(coefs) >
etudiants[meilleur].moyenne(coefs)) meilleur = j;
            }
            Etudiant tmp = etudiants[i];
            etudiants[i] = etudiants[meilleur];
            etudiants[meilleur] = tmp;
        }
    }
```

```
    void afficherClassement() {
        classer();
        int rang = 1;
        for (int i = 0; i < nb; i++) {
            double moy = etudiants[i].moyenne(coefs);
            // les ex aequo partagent le même rang : 1, 2, 2, 4…
            if (i > 0 && Math.abs(moy - etudiants[i - 1].moyenne(coefs)) > 1e-9)
rang = i + 1;
            System.out.printf("%2d%s  %-10s %6.2f  %s%n", rang, (rang == 1 ? "er" :
"e "), etudiants[i].getNom(), moy, mention(moy));
        }
    }

    void statistiquesParMatiere() {
        for (int m = 0; m < matieres.length; m++) {
            double somme = 0, max = -1;
            String major = "";
            for (int i = 0; i < nb; i++) {
                double n = etudiants[i].getNote(m);
                somme += n;
                if (n > max) { max = n; major = etudiants[i].getNom(); }
            }
            System.out.printf("%-14s moyenne %5.2f   meilleure note %5.2f (%s)%n",
matieres[m], somme / nb, max, major);
        }
    }

    double tauxReussite() {
        int admis = 0;
        for (int i = 0; i < nb; i++) if (etudiants[i].moyenne(coefs) >= 10)
admis++;
        return 100.0 * admis / nb;
    }
}

public class Bulletins {
    public static void main(String[] args) {
        String[] matieres = {"Mathématiques", "Java", "Anglais", "Réseaux"};
        double[] coefs = {4, 5, 2, 3};
        double[][] notes = {
            {14, 16, 12, 11}, {9, 8.5, 15, 10}, {17, 18, 14, 16},
            {12, 11, 9, 13}, {14, 16, 12, 11}, {6, 9, 11, 7}
        };
        String[] noms = {"Adjovi", "Bello", "Chabi", "Dossou", "Eyo", "Fagla"};

        Classe c = new Classe(matieres, coefs, 30);
        for (int i = 0; i < noms.length; i++) {
            Etudiant e = c.inscrire(noms[i]);
            for (int m = 0; m < matieres.length; m++) e.setNote(m, notes[i][m]);
        }
        c.afficherClassement();
        c.statistiquesParMatiere();
        System.out.printf("Taux de réussite : %.1f %%%n", c.tauxReussite());
        try {
            c.inscrire("Gbaguidi").setNote(0, 25);
        } catch (IllegalArgumentException ex) {
            System.out.println("Refusé : " + ex.getMessage());
        }
```

```
    }
}
```

A.12 TD 2 · Exercice 2 : gestion bancaire

Énoncé : section 9.2.

Points clés

- `retirer` est écrite une seule fois dans `Compte` et délègue la règle à `retraitAutorise` : c'est le patron de méthode (template method).
- Le crochet `apresRetrait`, vide par défaut, permet au compte épargne de compter ses retraits sans réécrire `retirer`.
- `virer` réutilise `retirer` : un virement sortant de l'épargne compte donc automatiquement comme un retrait, et rien n'est crédité si le débit échoue.
- `Agence.finDeMois()` appelle la même méthode sur tous les comptes ; chaque objet applique ses propres règles. Encours final : 114 460 F.

```
import java.util.ArrayList;

abstract class Compte {
    private static int prochainNumero = 1001;
    private final int numero;
    private final String titulaire;
    protected double solde;
    private final ArrayList<String> historique = new ArrayList<>();

    Compte(String titulaire, double depotInitial) {
        if (depotInitial < 0) throw new IllegalArgumentException("Dépôt initial
négatif");
        this.numero = prochainNumero++;
        this.titulaire = titulaire;
        this.solde = depotInitial;
        noter("Ouverture", depotInitial);
    }

    int getNumero() { return numero; }
    double getSolde() { return solde; }

    protected void noter(String libelle, double montant) {
        historique.add(String.format("%-22s %12.0f   solde %12.0f", libelle,
montant, solde));
    }

    void deposer(double montant) {
        if (montant <= 0) throw new IllegalArgumentException("Montant invalide");
        solde += montant;
        noter("Dépôt", montant);
    }

    // Chaque type de compte fixe ses propres règles de retrait
    protected abstract boolean retraitAutorise(double montant);

    boolean retirer(double montant) {
        if (montant <= 0 || !retraitAutorise(montant)) {
            noter("Retrait refusé", montant);
            return false;
        }
```

```
        solde -= montant;
        apresRetrait();
        noter("Retrait", -montant);
        return true;
    }

    protected void apresRetrait() { }

    // Le virement n'est crédité que si le débit a réussi
    boolean virer(Compte destinataire, double montant) {
        if (destinataire == this || !retirer(montant)) return false;
        destinataire.deposer(montant);
        return true;
    }

    abstract void finDeMois();

    void afficherReleve() {
        System.out.println(getClass().getSimpleName() + " n° " + numero + " (" +
titulaire + ")");
        for (String ligne : historique) System.out.println("   " + ligne);
    }
}

class CompteCourant extends Compte {
    private final double decouvertAutorise;
    static final double FRAIS_TENUE = 500, TAUX_AGIOS = 0.15;

    CompteCourant(String titulaire, double depot, double decouvert) {
        super(titulaire, depot);
        this.decouvertAutorise = decouvert;
    }

    @Override
    protected boolean retraitAutorise(double montant) {
        return solde - montant >= -decouvertAutorise;
    }

    @Override
    void finDeMois() {
        solde -= FRAIS_TENUE;
        noter("Frais de tenue", -FRAIS_TENUE);
        if (solde < 0) {
            double agios = -solde * TAUX_AGIOS / 12;
            solde -= agios;
            noter("Agios", -agios);
        }
    }
}

class CompteEpargne extends Compte {
    private final double tauxAnnuel;
    private int retraitsDuMois = 0;
    static final int MAX_RETRAITS = 2;

    CompteEpargne(String titulaire, double depot, double tauxAnnuel) {
        super(titulaire, depot);
        this.tauxAnnuel = tauxAnnuel;
    }

    @Override
```

```
    protected boolean retraitAutorise(double montant) {
        return retraitsDuMois < MAX_RETRAITS && montant <= solde;
    }

    @Override
    protected void apresRetrait() { retraitsDuMois++; }

    @Override
    void finDeMois() {
        double interets = solde * tauxAnnuel / 100 / 12;
        solde += interets;
        noter("Intérêts", interets);
        retraitsDuMois = 0;
    }
}

class Agence {
    private final ArrayList<Compte> comptes = new ArrayList<>();

    void ajouter(Compte c) { comptes.add(c); }

    Compte chercher(int numero) {
        for (Compte c : comptes) if (c.getNumero() == numero) return c;
        return null;
    }

    void finDeMois() {
        for (Compte c : comptes) c.finDeMois();     // polymorphisme : chaque
compte applique ses règles
    }

    double encoursTotal() {
        double total = 0;
        for (Compte c : comptes) total += c.getSolde();
        return total;
    }
}

public class BanqueAvancee {
    public static void main(String[] args) {
        Agence agence = new Agence();
        CompteCourant cc = new CompteCourant("Ahouansou", 50_000, 100_000);
        CompteEpargne ce = new CompteEpargne("Ahouansou", 200_000, 3.5);
        agence.ajouter(cc);
        agence.ajouter(ce);

        cc.retirer(120_000);                // accepté grâce au découvert
        cc.retirer(40_000);                 // refusé : dépasserait le découvert
        ce.retirer(10_000);
        ce.virer(cc, 30_000);               // deuxième retrait du mois sur
l'épargne
        ce.retirer(5_000);                  // refusé : plus de deux retraits
        agence.finDeMois();
        ce.retirer(5_000);                  // de nouveau autorisé le mois suivant

        cc.afficherReleve();
        ce.afficherReleve();
        System.out.printf("Encours total de l'agence : %.0f F%n",
agence.encoursTotal());
        System.out.println("Recherche du compte 1002 : " + (agence.chercher(1002)
!= null ? "trouvé" : "absent"));
```

```
    }
}
```

A.13 TD 2 · Exercice 3 : calcul matriciel

Énoncé : section 9.3.

Points clés

- Sans copie défensive, celui qui a fourni le tableau pourrait encore le modifier et changer la matrice « immuable ». La classe est `final` pour qu'aucune sous-classe ne puisse la rendre modifiable.
- L'exponentiation rapide demande environ $2\log_2(k)$ multiplications : une vingtaine pour $k = 1\,000$, contre 999 avec la méthode naïve.
- Le déterminant est développé selon la première ligne, avec des signes alternés ; la récursivité s'arrête aux matrices 2×2.
- Contrôles : $\det(A) = 24$, $\det(B) = -9$, $\det(AB) = -216$; $F(40) = 102\,334\,155$.

```
final class Matrice {
    private final double[][] m;
    private final int lignes, colonnes;

    Matrice(double[][] valeurs) {
        lignes = valeurs.length;
        colonnes = valeurs[0].length;
        m = new double[lignes][colonnes];
        for (int i = 0; i < lignes; i++) {
            if (valeurs[i].length != colonnes) throw new
IllegalArgumentException("Lignes de longueurs différentes");
            for (int j = 0; j < colonnes; j++) m[i][j] = valeurs[i][j];   // copie
défensive : la matrice reste immuable
        }
    }

    static Matrice identite(int n) {
        double[][] v = new double[n][n];
        for (int i = 0; i < n; i++) v[i][i] = 1;
        return new Matrice(v);
    }

    double get(int i, int j) { return m[i][j]; }

    Matrice plus(Matrice b) {
        if (lignes != b.lignes || colonnes != b.colonnes) throw new
IllegalArgumentException("Dimensions incompatibles");
        double[][] r = new double[lignes][colonnes];
        for (int i = 0; i < lignes; i++)
            for (int j = 0; j < colonnes; j++) r[i][j] = m[i][j] + b.m[i][j];
        return new Matrice(r);
    }

    Matrice fois(Matrice b) {
        if (colonnes != b.lignes) throw new IllegalArgumentException("Produit
impossible");
        double[][] r = new double[lignes][b.colonnes];
        for (int i = 0; i < lignes; i++)
            for (int j = 0; j < b.colonnes; j++)
                for (int k = 0; k < colonnes; k++) r[i][j] += m[i][k] * b.m[k][j];
```

```
        return new Matrice(r);
    }

    Matrice transposee() {
        double[][] r = new double[colonnes][lignes];
        for (int i = 0; i < lignes; i++)
            for (int j = 0; j < colonnes; j++) r[j][i] = m[i][j];
        return new Matrice(r);
    }

    // Exponentiation rapide : A^k = (A^(k/2))² si k pair, A × A^(k−1) sinon
    Matrice puissance(int k) {
        if (lignes != colonnes) throw new IllegalArgumentException("Matrice non
carrée");
        if (k == 0) return identite(lignes);
        if (k % 2 == 0) {
            Matrice moitie = puissance(k / 2);
            return moitie.fois(moitie);
        }
        return this.fois(puissance(k - 1));
    }

    // Sous-matrice obtenue en supprimant la ligne li et la colonne co
    private Matrice mineur(int li, int co) {
        double[][] r = new double[lignes - 1][colonnes - 1];
        for (int i = 0, ri = 0; i < lignes; i++) {
            if (i == li) continue;
            for (int j = 0, rj = 0; j < colonnes; j++) {
                if (j == co) continue;
                r[ri][rj++] = m[i][j];
            }
            ri++;
        }
        return new Matrice(r);
    }

    // Développement selon la première ligne (récursif)
    double determinant() {
        if (lignes != colonnes) throw new IllegalArgumentException("Matrice non
carrée");
        if (lignes == 1) return m[0][0];
        if (lignes == 2) return m[0][0] * m[1][1] - m[0][1] * m[1][0];
        double det = 0;
        for (int j = 0; j < colonnes; j++) {
            double signe = (j % 2 == 0) ? 1 : -1;
            det += signe * m[0][j] * mineur(0, j).determinant();
        }
        return det;
    }

    boolean egale(Matrice b) {
        if (lignes != b.lignes || colonnes != b.colonnes) return false;
        for (int i = 0; i < lignes; i++)
            for (int j = 0; j < colonnes; j++)
                if (Math.abs(m[i][j] - b.m[i][j]) > 1e-9) return false;
        return true;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
```

```
        for (double[] ligne : m) {
            for (double v : ligne) sb.append(String.format("%9.2f", v));
            sb.append("\n");
        }
        return sb.toString();
    }
}

public class Matrices {
    public static void main(String[] args) {
        Matrice a = new Matrice(new double[][]{{2, -1, 0}, {1, 3, 2}, {0, 1, 4}});
        Matrice b = new Matrice(new double[][]{{1, 0, 2}, {-1, 1, 0}, {3, 2, 1}});
        System.out.print("A × B =\n" + a.fois(b));
        System.out.println("A × I = A : " + a.fois(Matrice.identite(3)).egale(a));
        System.out.println("(AB)ᵀ = BᵀAᵀ : " +
a.fois(b).transposee().egale(b.transposee().fois(a.transposee())));
        System.out.printf("det(A) = %.1f   det(B) = %.1f   det(AB) = %.1f%n",
a.determinant(), b.determinant(), a.fois(b).determinant());
        Matrice fib = new Matrice(new double[][]{{1, 1}, {1, 0}});
        System.out.printf("Fibonacci : F(40) = %.0f (lu dans la matrice [[1,1],
[1,0]]^40)%n", fib.puissance(40).get(0, 1));
        try {
            a.plus(new Matrice(new double[][]{{1, 2}}));
        } catch (IllegalArgumentException e) {
            System.out.println("Refusé : " + e.getMessage());
        }
    }
}
```

A.14 TD 2 · Exercice 4 : agence de location de véhicules

Énoncé : section 9.4.

Points clés

- Le tarif dégressif est écrit une seule fois dans `Vehicule` ; chaque sous-classe ne fournit que son prix journalier et son intervalle de révision.
- `Camion` redéfinit `tarif` et appelle `super.tarif(Math.max(jours, 2))` : la remise reste calculée par la classe parente.
- `louer` renvoie −1 pour un véhicule introuvable, déjà loué ou à réviser ; l'agence manipule tous les véhicules à travers le type `Vehicule`.
- Contrôles : voiture 10 jours 225 000 F, camion 1 jour 176 000 F ; la moto revenue à 12 000 km ne peut plus être louée avant révision.

```
interface Louable {
    double tarif(int jours);
}

abstract class Vehicule implements Louable {
    private final String immatriculation;
    private long kilometrage;
    private long kmDerniereRevision;
    private boolean disponible = true;

    Vehicule(String immatriculation, long kilometrage) {
        this.immatriculation = immatriculation;
        this.kilometrage = kilometrage;
        this.kmDerniereRevision = kilometrage;
```

```
    }

    String getImmatriculation() { return immatriculation; }
    long getKilometrage() { return kilometrage; }
    boolean estDisponible() { return disponible; }

    abstract double prixJournalier();
    abstract long intervalleRevision();       // en kilomètres

    // Tarif dégressif commun à tous les véhicules
    @Override
    public double tarif(int jours) {
        double remise = (jours > 30) ? 0.25 : (jours > 7) ? 0.10 : 0;
        return jours * prixJournalier() * (1 - remise);
    }

    void sortir() {
        if (!disponible) throw new IllegalStateException(immatriculation + " est
déjà loué");
        disponible = false;
    }

    void rentrer(long kmParcourus) {
        if (disponible) throw new IllegalStateException(immatriculation + " n'était
pas loué");
        if (kmParcourus < 0) throw new IllegalArgumentException("Kilométrage
négatif");
        kilometrage += kmParcourus;
        disponible = true;
    }

    boolean doitEtreRevise() { return kilometrage - kmDerniereRevision >=
intervalleRevision(); }
    void reviser() { kmDerniereRevision = kilometrage; }

    @Override
    public String toString() {
        return String.format("%-8s %-10s %8d km  %s", getClass().getSimpleName(),
immatriculation, kilometrage,
                disponible ? "disponible" : "loué");
    }
}

class Voiture extends Vehicule {
    private final int places;
    Voiture(String immat, long km, int places) { super(immat, km); this.places =
places; }
    @Override double prixJournalier() { return places > 5 ? 35_000 : 25_000; }
    @Override long intervalleRevision() { return 15_000; }
}

class Camion extends Vehicule {
    private final double chargeTonnes;
    Camion(String immat, long km, double charge) { super(immat, km);
this.chargeTonnes = charge; }
    @Override double prixJournalier() { return 60_000 + 8_000 * chargeTonnes; }
    @Override long intervalleRevision() { return 30_000; }
    // Un camion se loue au minimum 2 jours
    @Override public double tarif(int jours) { return super.tarif(Math.max(jours,
2)); }
}
```

```
class Moto extends Vehicule {
    Moto(String immat, long km) { super(immat, km); }
    @Override double prixJournalier() { return 8_000; }
    @Override long intervalleRevision() { return 6_000; }
}

class AgenceLocation {
    private final Vehicule[] parc = new Vehicule[50];
    private int nb = 0;
    private double chiffreAffaires = 0;

    void ajouter(Vehicule v) { parc[nb++] = v; }

    private Vehicule trouver(String immat) {
        for (int i = 0; i < nb; i++) if
(parc[i].getImmatriculation().equals(immat)) return parc[i];
        return null;
    }

    double louer(String immat, int jours) {
        Vehicule v = trouver(immat);
        if (v == null || !v.estDisponible() || v.doitEtreRevise()) return -1;
        v.sortir();
        double montant = v.tarif(jours);
        chiffreAffaires += montant;
        return montant;
    }

    void rendre(String immat, long km) {
        Vehicule v = trouver(immat);
        if (v != null) v.rentrer(km);
    }

    // Tri par insertion, du plus fort au plus faible kilométrage
    void trierParKilometrage() {
        for (int i = 1; i < nb; i++) {
            Vehicule cle = parc[i];
            int j = i - 1;
            while (j >= 0 && parc[j].getKilometrage() < cle.getKilometrage()) {
                parc[j + 1] = parc[j];
                j--;
            }
            parc[j + 1] = cle;
        }
    }

    void afficher() {
        for (int i = 0; i < nb; i++) System.out.println("  " + parc[i] +
(parc[i].doitEtreRevise() ? "  [révision]" : ""));
    }

    double getChiffreAffaires() { return chiffreAffaires; }
}

public class ParcAuto {
    public static void main(String[] args) {
        AgenceLocation a = new AgenceLocation();
        a.ajouter(new Voiture("AB-123", 42_000, 5));
        a.ajouter(new Voiture("CD-456", 12_500, 7));
        a.ajouter(new Camion("EF-789", 88_000, 3.5));
```

```
        a.ajouter(new Moto("GH-012", 5_900));

        System.out.printf("Voiture 10 jours : %.0f F%n", a.louer("AB-123", 10));
        System.out.printf("Camion 1 jour (facturé 2) : %.0f F%n", a.louer("EF-789",
1));
        System.out.println("Voiture déjà louée : " + a.louer("AB-123", 3));
        a.rendre("AB-123", 1_200);
        a.rendre("EF-789", 450);
        System.out.printf("Moto 40 jours : %.0f F%n", a.louer("GH-012", 40));
        a.rendre("GH-012", 6_100);                    // plus de 6 000 km depuis
la dernière révision
        System.out.println("Moto à réviser, nouvelle location : " + a.louer("GH-
012", 1));
        a.trierParKilometrage();
        a.afficher();
        System.out.printf("Chiffre d'affaires : %.0f F%n", a.getChiffreAffaires());
    }
}
```

A.15 TD 2 · Exercice 5 : analyse de texte et cryptographie

Énoncé : section 9.5.

Points clés

- Le découpage ajoute un espace fictif après le dernier caractère pour ne pas oublier le dernier mot.
- En Java, `-3 % 26` vaut −3 : on écrit `((x % 26) + 26) % 26` pour obtenir un résultat toujours compris entre 0 et 25.
- Vigenère est un César dont le décalage change à chaque lettre ; le déchiffrement utilise les décalages opposés.
- La cryptanalyse par fréquence échoue sur un message trop court ou dont la lettre la plus fréquente n'est pas le E.

```
class AnalyseurTexte {
    private final String texte;

    AnalyseurTexte(String texte) { this.texte = texte; }

    // Découpe manuelle en mots (suite de lettres ou de chiffres), sans utiliser
split
    String[] mots() {
        String[] tmp = new String[texte.length()];
        int nb = 0;
        StringBuilder courant = new StringBuilder();
        for (int i = 0; i <= texte.length(); i++) {
            char c = (i < texte.length()) ? texte.charAt(i) : ' ';
            if (Character.isLetterOrDigit(c)) {
                courant.append(c);
            } else if (courant.length() > 0) {
                tmp[nb++] = courant.toString();
                courant.setLength(0);
            }
        }
        String[] res = new String[nb];
        for (int i = 0; i < nb; i++) res[i] = tmp[i];
        return res;
    }
```

```
    // Tableaux parallèles : mots distincts et nombre d'occurrences
    String motLePlusFrequent() {
        String[] m = mots();
        String[] distincts = new String[m.length];
        int[] compte = new int[m.length];
        int nb = 0;
        for (String mot : m) {
            String bas = mot.toLowerCase();
            int k = 0;
            while (k < nb && !distincts[k].equals(bas)) k++;
            if (k == nb) distincts[nb++] = bas;
            compte[k]++;
        }
        int max = 0;
        for (int k = 1; k < nb; k++) if (compte[k] > compte[max]) max = k;
        return nb == 0 ? "" : distincts[max] + " (" + compte[max] + " fois)";
    }

    String inverserMots() {
        String[] m = mots();
        StringBuilder sb = new StringBuilder();
        for (int i = m.length - 1; i >= 0; i--) sb.append(m[i]).append(i > 0 ? " "
: "");
        return sb.toString();
    }

    static boolean estPalindrome(String s) {
        String t = s.toLowerCase();
        int g = 0, d = t.length() - 1;
        while (g < d) {
            if (!Character.isLetter(t.charAt(g))) { g++; continue; }
            if (!Character.isLetter(t.charAt(d))) { d--; continue; }
            if (t.charAt(g) != t.charAt(d)) return false;
            g++;
            d--;
        }
        return true;
    }

    static boolean sontAnagrammes(String a, String b) {
        int[] compte = new int[26];
        for (char c : a.toLowerCase().toCharArray()) if (c >= 'a' && c <= 'z')
compte[c - 'a']++;
        for (char c : b.toLowerCase().toCharArray()) if (c >= 'a' && c <= 'z')
compte[c - 'a']--;
        for (int v : compte) if (v != 0) return false;
        return true;
    }

    private static char decaler(char c, int decalage) {
        if (c >= 'a' && c <= 'z') return (char) ('a' + ((c - 'a' + decalage) % 26 +
26) % 26);
        if (c >= 'A' && c <= 'Z') return (char) ('A' + ((c - 'A' + decalage) % 26 +
26) % 26);
        return c;
    }

    static String cesar(String s, int decalage) {
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray()) sb.append(decaler(c, decalage));
```

```
        return sb.toString();
    }

    // sens = +1 pour chiffrer, -1 pour déchiffrer ; la clé n'avance que sur les
lettres
    static String vigenere(String s, String cle, int sens) {
        StringBuilder sb = new StringBuilder();
        int k = 0;
        for (char c : s.toCharArray()) {
            if (Character.isLetter(c)) {
                int d = Character.toUpperCase(cle.charAt(k % cle.length())) - 'A';
                sb.append(decaler(c, sens * d));
                k++;
            } else {
                sb.append(c);
            }
        }
        return sb.toString();
    }

    // Cassage de César : on suppose que la lettre la plus fréquente du message
chiffré correspond au E
    static int devinerDecalage(String chiffre) {
        int[] freq = new int[26];
        for (char c : chiffre.toUpperCase().toCharArray()) if (c >= 'A' && c <=
'Z') freq[c - 'A']++;
        int max = 0;
        for (int i = 1; i < 26; i++) if (freq[i] > freq[max]) max = i;
        return ((max - ('E' - 'A')) % 26 + 26) % 26;
    }
}

public class Texte {
    public static void main(String[] args) {
        AnalyseurTexte t = new AnalyseurTexte("Le chat dort.  Le chien joue, et le
chat regarde le chien !");
        System.out.println("Nombre de mots : " + t.mots().length);
        System.out.println("Mot le plus fréquent : " + t.motLePlusFrequent());
        System.out.println("Mots inversés : " + t.inverserMots());
        System.out.println("Palindrome ? " + AnalyseurTexte.estPalindrome("Esope
reste ici et se repose"));
        System.out.println("Anagrammes ? " + AnalyseurTexte.sontAnagrammes("Marie",
"aimer") + " / "
                + AnalyseurTexte.sontAnagrammes("Java", "vaja!") + " / " +
AnalyseurTexte.sontAnagrammes("Java", "Jave"));
        String c = AnalyseurTexte.cesar("Rendez-vous demain a Cotonou, venez seuls
et en avance", 7);
        System.out.println("César : " + c);
        int d = AnalyseurTexte.devinerDecalage(c);
        System.out.println("Décalage deviné : " + d + " -> " +
AnalyseurTexte.cesar(c, -d));
        String v = AnalyseurTexte.vigenere("BONJOUR", "CLE", 1);
        System.out.println("Vigenère : " + v + " -> " + AnalyseurTexte.vigenere(v,
"CLE", -1));
    }
}
```

A.16 TD 2 · Exercice 6 : morpion contre l'ordinateur

Énoncé : section 9.6.

Points clés

- L'ordinateur teste chaque coup en le jouant puis en l'annulant : le plateau est toujours remis dans son état initial.
- Une fourchette est un coup après lequel il existe au moins deux cases gagnantes : l'adversaire ne peut en bloquer qu'une.
- `partie` ne connaît que le type `Joueur` : grâce à la liaison dynamique, la même boucle fait jouer humains et ordinateurs.
- Deux ordinateurs font toujours match nul, ce qui est le résultat d'une partie parfaitement jouée des deux côtés.

```
import java.util.Scanner;

class Plateau {
    private final char[][] cases = new char[3][3];

    Plateau() {
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++) cases[i][j] = ' ';
    }

    boolean estLibre(int l, int c) { return l >= 0 && l < 3 && c >= 0 && c < 3 &&
cases[l][c] == ' '; }

    boolean jouer(int l, int c, char symbole) {
        if (!estLibre(l, c)) return false;
        cases[l][c] = symbole;
        return true;
    }

    void annuler(int l, int c) { cases[l][c] = ' '; }

    // Renvoie le symbole gagnant, ou ' ' s'il n'y en a pas
    char gagnant() {
        for (int i = 0; i < 3; i++) {
            if (cases[i][0] != ' ' && cases[i][0] == cases[i][1] && cases[i][1] ==
cases[i][2]) return cases[i][0];
            if (cases[0][i] != ' ' && cases[0][i] == cases[1][i] && cases[1][i] ==
cases[2][i]) return cases[0][i];
        }
        if (cases[1][1] != ' ' && ((cases[0][0] == cases[1][1] && cases[1][1] ==
cases[2][2])
                || (cases[0][2] == cases[1][1] && cases[1][1] == cases[2][0])))
return cases[1][1];
        return ' ';
    }

    boolean estPlein() {
        for (char[] ligne : cases) for (char c : ligne) if (c == ' ') return false;
        return true;
    }

    void afficher() {
        System.out.println("    1   2   3");
        for (int i = 0; i < 3; i++) {
```

```
            System.out.println((char) ('A' + i) + "   " + cases[i][0] + " | " +
cases[i][1] + " | " + cases[i][2]);
            if (i < 2) System.out.println("   ---+---+---");
        }
    }
}

abstract class Joueur {
    protected final String nom;
    protected final char symbole;

    Joueur(String nom, char symbole) { this.nom = nom; this.symbole = symbole; }

    abstract int[] choisirCoup(Plateau p);
}

class Humain extends Joueur {
    private final Scanner sc;
    Humain(String nom, char symbole, Scanner sc) { super(nom, symbole); this.sc =
sc; }

    @Override
    int[] choisirCoup(Plateau p) {
        while (true) {
            System.out.print(nom + " (" + symbole + "), votre case (ex. B2) : ");
            String s = sc.next().toUpperCase();
            if (s.length() == 2) {
                int l = s.charAt(0) - 'A', c = s.charAt(1) - '1';
                if (p.estLibre(l, c)) return new int[]{l, c};
            }
            System.out.println("Case invalide ou occupée.");
        }
    }
}

class Ordinateur extends Joueur {
    private final char adversaire;
    Ordinateur(String nom, char symbole, char adversaire) { super(nom, symbole);
this.adversaire = adversaire; }

    // Cherche une case qui donnerait la victoire au symbole indiqué
    private int[] coupGagnant(Plateau p, char s) {
        for (int l = 0; l < 3; l++)
            for (int c = 0; c < 3; c++)
                if (p.estLibre(l, c)) {
                    p.jouer(l, c, s);
                    boolean gagne = p.gagnant() == s;
                    p.annuler(l, c);
                    if (gagne) return new int[]{l, c};
                }
        return null;
    }

    // Une fourchette crée deux menaces de victoire à la fois
    private int nbMenaces(Plateau p, char s) {
        int n = 0;
        for (int l = 0; l < 3; l++)
            for (int c = 0; c < 3; c++)
                if (p.estLibre(l, c)) {
                    p.jouer(l, c, s);
                    if (p.gagnant() == s) n++;
```

```
                    p.annuler(l, c);
                }
        return n;
    }

    private int[] fourchette(Plateau p, char s) {
        for (int l = 0; l < 3; l++)
            for (int c = 0; c < 3; c++)
                if (p.estLibre(l, c)) {
                    p.jouer(l, c, s);
                    boolean ok = nbMenaces(p, s) >= 2;
                    p.annuler(l, c);
                    if (ok) return new int[]{l, c};
                }
        return null;
    }

    @Override
    int[] choisirCoup(Plateau p) {
        int[] coup = coupGagnant(p, symbole);                // 1. gagner
        if (coup == null) coup = coupGagnant(p, adversaire);  // 2. bloquer
        if (coup == null) coup = fourchette(p, symbole);      // 3. créer une
fourchette
        if (coup == null && fourchette(p, adversaire) != null) {
            // 4. contrer une fourchette adverse en jouant un bord (force
l'adversaire à défendre)
            int[][] bords = {{0, 1}, {1, 0}, {1, 2}, {2, 1}};
            for (int[] b : bords) if (p.estLibre(b[0], b[1])) { coup = b; break; }
        }
        if (coup == null && p.estLibre(1, 1)) coup = new int[]{1, 1};   // 5.
centre
        int[][] ordre = {{0, 0}, {0, 2}, {2, 0}, {2, 2}, {0, 1}, {1, 0}, {1, 2},
{2, 1}};
        for (int i = 0; coup == null && i < ordre.length; i++)         // 6. coins
puis bords
            if (p.estLibre(ordre[i][0], ordre[i][1])) coup = ordre[i];
        return coup;
    }
}

public class Morpion {
    // Renvoie le gagnant, ou null en cas de match nul
    static Joueur partie(Joueur j1, Joueur j2, boolean afficher) {
        Plateau p = new Plateau();
        Joueur courant = j1;
        while (true) {
            if (afficher) p.afficher();
            int[] coup = courant.choisirCoup(p);
            p.jouer(coup[0], coup[1], courant.symbole);
            if (p.gagnant() == courant.symbole) {
                if (afficher) p.afficher();
                return courant;
            }
            if (p.estPlein()) {
                if (afficher) p.afficher();
                return null;
            }
            courant = (courant == j1) ? j2 : j1;
        }
    }
```

```
    public static void main(String[] args) {
        // Test : deux ordinateurs qui jouent parfaitement font toujours match nul
        Joueur o1 = new Ordinateur("Ordi 1", 'X', 'O'), o2 = new Ordinateur("Ordi
2", 'O', 'X');
        Joueur g = partie(o1, o2, false);
        System.out.println("Ordinateur contre ordinateur : " + (g == null ? "match
nul" : g.nom + " gagne"));

        Scanner sc = new Scanner(System.in);
        Joueur humain = new Humain("Joueur", 'X', sc);
        Joueur ordi = new Ordinateur("Ordinateur", 'O', 'X');
        Joueur v = partie(humain, ordi, true);
        System.out.println(v == null ? "Match nul." : v.nom + " a gagné !");
        sc.close();
    }
}
```

A.17 TD 2 · Exercice 7 : gestion de commandes

Énoncé : section 9.7.

Points clés

- `ProduitPerissable` ne redéfinit que `prixUnitaire()` ; les lignes et la commande l'utilisent sans connaître le type réel du produit.
- `ajouter` cumule les quantités déjà commandées pour un même produit avant de les comparer au stock.
- La validation vérifie toutes les lignes avant de déstocker : si l'on déstockait au fil de l'eau, un échec à la troisième ligne laisserait les deux premières sorties du stock sans commande valide.
- Contrôle : total brut 597 000 F, remise 10 %, net à payer 634 014 F.

```
import java.util.ArrayList;

class Produit {
    private final String reference, designation;
    private final double prixHT;
    private int stock;

    Produit(String reference, String designation, double prixHT, int stock) {
        if (prixHT <= 0 || stock < 0) throw new IllegalArgumentException("Prix ou
stock invalide");
        this.reference = reference;
        this.designation = designation;
        this.prixHT = prixHT;
        this.stock = stock;
    }

    String getReference() { return reference; }
    String getDesignation() { return designation; }
    int getStock() { return stock; }
    double getPrixHT() { return prixHT; }

    // Prix réellement appliqué ; les sous-classes peuvent le modifier
    double prixUnitaire() { return prixHT; }

    void sortirDuStock(int qte) {
        if (qte > stock) throw new IllegalStateException("Stock insuffisant pour "
```

```
+ reference);
        stock -= qte;
    }
}

class ProduitPerissable extends Produit {
    private final int joursAvantPeremption;

    ProduitPerissable(String ref, String des, double prix, int stock, int jours) {
        super(ref, des, prix, stock);
        this.joursAvantPeremption = jours;
    }

    boolean estPerime() { return joursAvantPeremption <= 0; }

    @Override
    double prixUnitaire() {
        if (joursAvantPeremption <= 2) return getPrixHT() * 0.5;
        if (joursAvantPeremption <= 5) return getPrixHT() * 0.8;
        return getPrixHT();
    }
}

class LigneCommande {
    private final Produit produit;
    private final int quantite;

    LigneCommande(Produit produit, int quantite) {
        this.produit = produit;
        this.quantite = quantite;
    }

    Produit getProduit() { return produit; }
    int getQuantite() { return quantite; }
    double sousTotal() { return produit.prixUnitaire() * quantite; }
}

class Commande {
    private static int compteur = 0;
    private final int numero;
    private final String client;
    private final ArrayList<LigneCommande> lignes = new ArrayList<>();
    private boolean validee = false;
    static final double TVA = 0.18;

    Commande(String client) {
        this.numero = ++compteur;
        this.client = client;
    }

    // Si le produit est déjà commandé, la quantité totale doit rester disponible
    boolean ajouter(Produit p, int qte) {
        if (validee || qte <= 0) return false;
        if (p instanceof ProduitPerissable pp && pp.estPerime()) return false;
        int dejaCommande = 0;
        for (LigneCommande l : lignes) if (l.getProduit() == p) dejaCommande +=
l.getQuantite();
        if (dejaCommande + qte > p.getStock()) return false;
        lignes.add(new LigneCommande(p, qte));
        return true;
    }
```

```
    double totalBrutHT() {
        double t = 0;
        for (LigneCommande l : lignes) t += l.sousTotal();
        return t;
    }

    double tauxRemise() {
        double brut = totalBrutHT();
        if (brut >= 500_000) return 0.10;
        if (brut >= 100_000) return 0.05;
        return 0;
    }

    // La validation est tout ou rien : on vérifie d'abord toutes les lignes, puis
on déstocke
    boolean valider() {
        if (validee || lignes.isEmpty()) return false;
        for (LigneCommande l : lignes) if (l.getQuantite() >
l.getProduit().getStock()) return false;
        for (LigneCommande l : lignes)
l.getProduit().sortirDuStock(l.getQuantite());
        validee = true;
        return true;
    }

    void imprimerFacture() {
        System.out.println("Facture n° " + numero + ", client : " + client +
(validee ? "" : " (NON VALIDÉE)"));
        for (LigneCommande l : lignes) {
            System.out.printf("  %-6s %-18s %4d x %9.0f = %11.0f%n",
l.getProduit().getReference(),
                    l.getProduit().getDesignation(), l.getQuantite(),
l.getProduit().prixUnitaire(), l.sousTotal());
        }
        double brut = totalBrutHT(), remise = brut * tauxRemise(), ht = brut -
remise, tva = ht * TVA;
        System.out.printf("  Total brut HT %38.0f%n  Remise (%2.0f %%) %36.0f%n
Total HT %43.0f%n  TVA 18 %% %43.0f%n  Net à payer %40.0f%n",
                brut, tauxRemise() * 100, -remise, ht, tva, ht + tva);
    }
}

public class Commandes {
    public static void main(String[] args) {
        Produit clavier = new Produit("CLV01", "Clavier USB", 7_500, 40);
        Produit ecran = new Produit("ECR27", "Écran 27 pouces", 145_000, 3);
        ProduitPerissable yaourt = new ProduitPerissable("YAO12", "Yaourt (pack)",
2_400, 100, 2);
        ProduitPerissable lait = new ProduitPerissable("LAI01", "Lait (périmé)",
900, 50, 0);

        Commande c1 = new Commande("Lycée Behanzin");
        System.out.println("Clavier x 20 : " + c1.ajouter(clavier, 20));
        System.out.println("Écran x 4 : " + c1.ajouter(ecran, 4) + " (stock : 3)");
        System.out.println("Écran x 3 : " + c1.ajouter(ecran, 3));
        System.out.println("Yaourt x 10 : " + c1.ajouter(yaourt, 10));
        System.out.println("Lait périmé : " + c1.ajouter(lait, 5));
        System.out.println("Validation : " + c1.valider());
        c1.imprimerFacture();
```

```
        Commande c2 = new Commande("Boutique Zinsou");
        System.out.println("Écran après validation de c1 : " + c2.ajouter(ecran, 1)
+ " (stock : " + ecran.getStock() + ")");
    }
}
```

A.18 TD 2 · Exercice 8 : simulation de file d'attente

Énoncé : section 9.8.

Points clés

- Dans la file circulaire, la position d'insertion est `(debut + taille) % capacite` : enfiler et défiler se font en temps constant, sans décaler les éléments.
- La file mémorise la minute d'arrivée de chaque client ; l'attente se calcule au moment où un guichet le prend.
- La même graine produit la même suite d'arrivées pour les trois scénarios : seules les conditions de service changent, la comparaison est équitable.
- Avec 0,3 arrivée par minute et 4 minutes de service en moyenne, un guichet doit traiter 1,2 minute de travail par minute : il est saturé et la file grandit (32,7 min d'attente en moyenne avec la graine 2022). Deux guichets ne sont occupés qu'à 60 % : l'attente tombe à 0,4 min.

```
import java.util.Random;

// File circulaire d'entiers implémentée avec un tableau (on y range l'heure
d'arrivée de chaque client)
class FileCirculaire {
    private final int[] t;
    private int debut = 0, taille = 0;

    FileCirculaire(int capacite) { t = new int[capacite]; }

    boolean estVide() { return taille == 0; }
    boolean estPleine() { return taille == t.length; }
    int taille() { return taille; }

    void enfiler(int v) {
        if (estPleine()) throw new IllegalStateException("File pleine");
        t[(debut + taille) % t.length] = v;     // la fin « boucle » au début du
tableau
        taille++;
    }

    int defiler() {
        if (estVide()) throw new IllegalStateException("File vide");
        int v = t[debut];
        debut = (debut + 1) % t.length;
        taille--;
        return v;
    }
}

class Simulation {
    private final int nbGuichets, duree, capacite;
    private final double probaArrivee;
    private final Random hasard;

    int servis = 0, refuses = 0, attenteMax = 0, fileMax = 0;
```

```
    long attenteTotale = 0;

    Simulation(int nbGuichets, double probaArrivee, int duree, int capacite, long
graine) {
        this.nbGuichets = nbGuichets;
        this.probaArrivee = probaArrivee;
        this.duree = duree;
        this.capacite = capacite;
        this.hasard = new Random(graine);     // même graine = même suite
d'arrivées : comparaison équitable
    }

    void executer() {
        FileCirculaire file = new FileCirculaire(capacite);
        int[] occupeJusqua = new int[nbGuichets];    // minute à laquelle chaque
guichet se libère
        for (int minute = 0; minute < duree; minute++) {
            if (hasard.nextDouble() < probaArrivee) {
                if (file.estPleine()) refuses++;
                else file.enfiler(minute);
            }
            fileMax = Math.max(fileMax, file.taille());
            for (int g = 0; g < nbGuichets; g++) {
                if (occupeJusqua[g] <= minute && !file.estVide()) {
                    int arrivee = file.defiler();
                    int attente = minute - arrivee;
                    attenteTotale += attente;
                    attenteMax = Math.max(attenteMax, attente);
                    servis++;
                    occupeJusqua[g] = minute + 2 + hasard.nextInt(5);   // service
de 2 à 6 minutes
                }
            }
        }
    }

    void rapport() {
        System.out.printf("%d guichet(s) : %3d servis, %3d refusés, attente moyenne
%5.1f min, max %3d min, file max %2d%n",
                nbGuichets, servis, refuses, servis == 0 ? 0 : (double)
attenteTotale / servis, attenteMax, fileMax);
    }
}

public class Guichets {
    public static void main(String[] args) {
        FileCirculaire f = new FileCirculaire(3);
        f.enfiler(1); f.enfiler(2); f.enfiler(3);
        f.defiler(); f.enfiler(4);                   // 4 est rangé dans la case
libérée au début du tableau
        System.out.println("Test de la file : " + f.defiler() + " " + f.defiler() +
" " + f.defiler());

        for (int g = 1; g <= 3; g++) {
            Simulation s = new Simulation(g, 0.3, 480, 30, 2022);
            s.executer();
            s.rapport();
        }
    }
}
```

A.19 TD 2 · Exercice 9 : bataille navale

Énoncé : section 9.9.

Points clés

- Le placement aléatoire tire des positions jusqu'à en trouver une qui ne sort pas de la grille et ne chevauche aucun navire.
- `lireCase` convertit la lettre avec `charAt(0) - 'A'` et le nombre avec `Integer.parseInt` ; toute saisie hors grille renvoie `null`.
- Le tireur automatique vise d'abord les cases où `(l + c)` est pair : comme tout navire occupe au moins deux cases voisines, il en possède forcément une de cette couleur du damier.
- Avec la graine 7, le tireur automatique coule la flotte en 97 tirs.

```
import java.util.Random;

class Navire {
    private final String nom;
    private final int taille;
    private int ligne, colonne, touches = 0;
    private boolean horizontal;

    Navire(String nom, int taille) { this.nom = nom; this.taille = taille; }

    String getNom() { return nom; }
    int getTaille() { return taille; }

    void positionner(int l, int c, boolean h) { ligne = l; colonne = c; horizontal
= h; }

    boolean occupe(int l, int c) {
        return horizontal ? (l == ligne && c >= colonne && c < colonne + taille)
                          : (c == colonne && l >= ligne && l < ligne + taille);
    }

    void toucher() { touches++; }
    boolean estCoule() { return touches == taille; }
}

class Grille {
    static final int N = 10;
    private final Navire[][] cases = new Navire[N][N];
    private final boolean[][] tirs = new boolean[N][N];
    private final Navire[] flotte;

    Grille(Navire[] flotte) { this.flotte = flotte; }

    private boolean peutPlacer(Navire n, int l, int c, boolean h) {
        for (int k = 0; k < n.getTaille(); k++) {
            int li = h ? l : l + k, co = h ? c + k : c;
            if (li >= N || co >= N || cases[li][co] != null) return false;
        }
        return true;
    }

    // Placement aléatoire : on tire des positions jusqu'à en trouver une valide
pour chaque navire
    void placerAleatoirement(Random r) {
        for (Navire n : flotte) {
```

```
            int l, c;
            boolean h;
            do {
                h = r.nextBoolean();
                l = r.nextInt(N);
                c = r.nextInt(N);
            } while (!peutPlacer(n, l, c, h));
            n.positionner(l, c, h);
            for (int k = 0; k < n.getTaille(); k++) cases[h ? l : l + k][h ? c + k
: c] = n;
        }
    }

    // Convertit "B7" en indices ; renvoie null si la saisie est invalide
    static int[] lireCase(String s) {
        s = s.trim().toUpperCase();
        if (s.length() < 2 || s.length() > 3) return null;
        int l = s.charAt(0) - 'A';
        int c;
        try {
            c = Integer.parseInt(s.substring(1)) - 1;
        } catch (NumberFormatException e) {
            return null;
        }
        return (l >= 0 && l < N && c >= 0 && c < N) ? new int[]{l, c} : null;
    }

    String tirer(int l, int c) {
        if (tirs[l][c]) return "Déjà joué";
        tirs[l][c] = true;
        Navire n = cases[l][c];
        if (n == null) return "À l'eau";
        n.toucher();
        return n.estCoule() ? "Touché-coulé : " + n.getNom() : "Touché";
    }

    boolean flotteCoulee() {
        for (Navire n : flotte) if (!n.estCoule()) return false;
        return true;
    }

    void afficher(boolean montrerNavires) {
        System.out.println("   1 2 3 4 5 6 7 8 9 10");
        for (int l = 0; l < N; l++) {
            StringBuilder sb = new StringBuilder().append((char) ('A' +
l)).append("  ");
            for (int c = 0; c < N; c++) {
                char s = '.';
                if (tirs[l][c]) s = (cases[l][c] != null) ? 'X' : 'o';
                else if (montrerNavires && cases[l][c] != null) s = '#';
                sb.append(s).append(' ');
            }
            System.out.println(sb);
        }
    }
}

public class BatailleNavale {
    static Navire[] flotte() {
        return new Navire[]{new Navire("Porte-avions", 5), new Navire("Croiseur",
4),
```

```
                new Navire("Contre-torpilleur", 3), new Navire("Sous-marin", 3),
new Navire("Torpilleur", 2)};
    }

    public static void main(String[] args) {
        Grille g = new Grille(flotte());
        g.placerAleatoirement(new Random(7));
        g.afficher(true);

        System.out.println("Saisies : B7 -> " +
java.util.Arrays.toString(Grille.lireCase("B7"))
                + ", J10 -> " + java.util.Arrays.toString(Grille.lireCase("J10")) +
", K3 -> " + Grille.lireCase("K3"));

        // Tireur automatique de test : une case sur deux d'abord (tout navire y
est touché), puis les autres
        int tirs = 0;
        for (int passe = 0; passe < 2 && !g.flotteCoulee(); passe++)
            for (int l = 0; l < Grille.N && !g.flotteCoulee(); l++)
                for (int c = 0; c < Grille.N && !g.flotteCoulee(); c++)
                    if ((l + c) % 2 == passe) {
                        String r = g.tirer(l, c);
                        tirs++;
                        if (r.startsWith("Touché-coulé")) System.out.println((char)
('A' + l) + "" + (c + 1) + " : " + r);
                    }
        System.out.println("Flotte coulée en " + tirs + " tirs.");
        g.afficher(false);
    }
}
```

A.20 TD 2 · Exercice 10 : paie et organigramme

Énoncé : section 9.10.

Points clés

- L'impôt progressif parcourt les tranches : chacune ne taxe que la partie du salaire comprise entre ses bornes. Sur 200 000 F : 9 000 + 7 500 = 16 500 F.
- `Stagiaire` redéfinit `impot()` et `salaireNet()` ; `Commercial` réutilise le calcul du salarié par `super.salaireBrut()`.
- `Manager` est récursif : sa masse d'équipe additionne les masses d'équipe de ses subordonnés, qui peuvent eux-mêmes être des managers (structure composite).
- Aucun test de type n'est nécessaire : chaque objet répond avec sa propre version de la méthode. Masse salariale totale : 3 249 427 F.

```
abstract class Employe {
    private static int compteur = 0;
    private final int matricule;
    private final String nom;

    Employe(String nom) {
        this.nom = nom;
        this.matricule = ++compteur;
    }

    String getNom() { return nom; }
    int getMatricule() { return matricule; }
```

```
    abstract double salaireBrut();

    static final double TAUX_CNSS = 0.036;

    // Impôt mensuel progressif : chaque tranche n'est taxée qu'à son propre taux
    double impot() {
        double b = salaireBrut(), impot = 0;
        double[] bornes = {60_000, 150_000, 300_000};
        double[] taux = {0.0, 0.10, 0.15, 0.25};
        double bas = 0;
        for (int i = 0; i <= bornes.length; i++) {
            double haut = (i < bornes.length) ? bornes[i] : Double.MAX_VALUE;
            if (b > bas) impot += (Math.min(b, haut) - bas) * taux[i];
            bas = haut;
        }
        return impot;
    }

    double salaireNet() { return salaireBrut() * (1 - TAUX_CNSS) - impot(); }

    // Masse salariale brute de l'employé et de toute son équipe (redéfinie par
Manager)
    double masseEquipe() { return salaireBrut(); }

    void afficherOrganigramme(int niveau) {
        String retrait = "";
        for (int i = 0; i < niveau; i++) retrait += "    ";
        System.out.printf("%s%-14s %-10s brut %9.0f   net %9.0f%n", retrait,
getNom(), getClass().getSimpleName(),
                salaireBrut(), salaireNet());
    }

    // Employé de plus haut salaire net dans la sous-arborescence (redéfinie par
Manager)
    Employe mieuxPaye() { return this; }
}

class Salarie extends Employe {
    private final double base;
    private final int anciennete;

    Salarie(String nom, double base, int anciennete) {
        super(nom);
        this.base = base;
        this.anciennete = anciennete;
    }

    @Override
    double salaireBrut() { return base * (1 + Math.min(0.02 * anciennete, 0.30)); }
}

class Commercial extends Salarie {
    private double chiffreAffaires;

    Commercial(String nom, double base, int anciennete, double ca) {
        super(nom, base, anciennete);
        this.chiffreAffaires = ca;
    }

    @Override
```

```
    double salaireBrut() { return super.salaireBrut() + 0.04 * chiffreAffaires; }
}

class Stagiaire extends Employe {
    private final double gratification;
    Stagiaire(String nom, double gratification) { super(nom); this.gratification =
gratification; }
    @Override double salaireBrut() { return gratification; }
    @Override double impot() { return 0; }                  // gratification non
imposable
    @Override double salaireNet() { return gratification; }  // et sans cotisation
}

class Manager extends Salarie {
    private final Employe[] equipe = new Employe[20];
    private int nb = 0;
    static final double PRIME_PAR_SUBORDONNE = 5_000;

    Manager(String nom, double base, int anciennete) { super(nom, base,
anciennete); }

    void ajouter(Employe e) {
        if (e == this) throw new IllegalArgumentException("Un manager ne peut pas
se diriger lui-même");
        equipe[nb++] = e;
    }

    // Masse brute de toute l'équipe, sous-équipes comprises, sans compter le
manager lui-même
    private double masseSubordonnes() {
        double total = 0;
        for (int i = 0; i < nb; i++) total += equipe[i].masseEquipe();
        return total;
    }

    @Override
    double salaireBrut() {
        return super.salaireBrut() + nb * PRIME_PAR_SUBORDONNE + 0.01 *
masseSubordonnes();
    }

    @Override
    double masseEquipe() { return salaireBrut() + masseSubordonnes(); }

    @Override
    void afficherOrganigramme(int niveau) {
        super.afficherOrganigramme(niveau);
        for (int i = 0; i < nb; i++) equipe[i].afficherOrganigramme(niveau + 1);
    }

    @Override
    Employe mieuxPaye() {
        Employe meilleur = this;
        for (int i = 0; i < nb; i++) {
            Employe e = equipe[i].mieuxPaye();
            if (e.salaireNet() > meilleur.salaireNet()) meilleur = e;
        }
        return meilleur;
    }
}
```

```
public class Organigramme {
    public static void main(String[] args) {
        Manager dg = new Manager("Mme Hounkpè", 600_000, 12);
        Manager dirCom = new Manager("M. Assogba", 400_000, 8);
        Manager dirTech = new Manager("Mme Kiki", 450_000, 5);
        dg.ajouter(dirCom);
        dg.ajouter(dirTech);
        dirCom.ajouter(new Commercial("Codjo", 150_000, 3, 9_000_000));
        dirCom.ajouter(new Commercial("Sènan", 150_000, 1, 2_500_000));
        dirTech.ajouter(new Salarie("Ayaba", 300_000, 20));
        dirTech.ajouter(new Salarie("Fifamè", 250_000, 2));
        dirTech.ajouter(new Stagiaire("Gildas", 50_000));

        dg.afficherOrganigramme(0);
        System.out.printf("Masse salariale brute totale : %.0f F%n",
dg.masseEquipe());
        Employe m = dg.mieuxPaye();
        System.out.printf("Salaire net le plus élevé : %s (%.0f F)%n", m.getNom(),
m.salaireNet());
        System.out.printf("Impôt sur un brut de 200 000 F : %.0f F%n", new
Salarie("Test", 200_000, 0).impot());
    }
}
```